Programação em C#



Profº Elton Rodrigo

Esta apostila foi criada e editada com base em informações contidas nos sites imasters.com.br, alura.com.br, codigofonte.net, learn.microsoft.com, dotnet.microsoft.com, linhadecodigo.com.br, scriptbrasil.com.br e devmedia.com.

As imagens nela contidas foram capturadas com *PrintScreen* de computadores associados.

Sumário

Introdução **4**

1. Conhecendo a linguagem **5**

1.1. O que é C# 5

1.2. Uma sintaxe simples 6

1.2.1. Criando um projeto 6

2. Variáveis e tipos de dados **11**

2.1. Variáveis 11

2.1.1. Regras para nomes de variáveis 11

2.2. Tipos de dados 12

3. Programando no Console .NET **14**

3.1. Escrevendo e Lendo Dados no Console .NET 14

3.1.1. Conversão de dados 15

4. Estruturas de controle **17**

4.1. Tomando decisões no código 17

4.2. Mais sobre condições 18

5. Windows Forms **20**

5.1. Criando um aplicativo Windows Forms 20

5.2. Adicionando Controles ao Formulário 24

5.3. Explorando os Arquivos .cs e .Designer.cs 25

5.3.1. Atribuindo Nomes aos Controles 25

5.3.2. Atribuindo Comandos ao Botão 26

5.4. Convertendo o Valor do TextBox em uma Variável 28

6. Banco de Dados **30**

6.1. Criando e configurando um conjunto Banco de Dados 30

6.2. Criando tabelas 32

6.3. Exibir conteúdo de tabelas em um Web Form 36

6.4. Organizando o código 38

6.5. Inserir registros a partir de um formulário 43

6.6. Selecionar registros pela GridView 45

6.7. Exclusão de registros 50

6.8. Alteração de registros 52

Introdução

Bem-vindo a programação em **C#** com **Microsoft Visual Studio**! Nesta apostila, vamos explorar desde os conceitos fundamentais até os tópicos mais avançados, capacitando você a criar aplicativos tanto para console quanto para interfaces gráficas usando o Microsoft Visual Studio.

Começaremos abordando os conceitos básicos da linguagem C#, incluindo variáveis, estruturas de controle e funções. Em seguida, avançaremos para a programação orientada a objetos, explorando classes, herança, polimorfismo e encapsulamento.

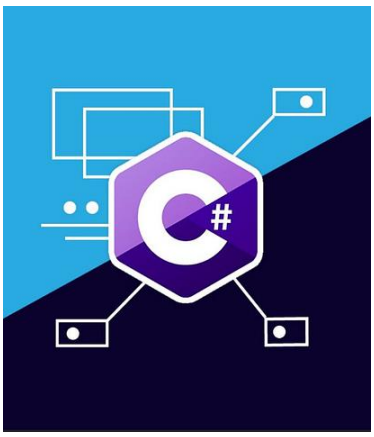
No início, focaremos na criação de aplicativos de console, ideais para entender os fundamentos da linguagem e praticar os conceitos aprendidos. Você desenvolverá aplicativos interativos, aprendendo a manipular entrada e saída de dados e a utilizar estruturas de controle para criar lógica de programa eficiente.

Em seguida, mergulharemos na criação de aplicativos com interface gráfica de usuário (GUI) utilizando Windows Forms no .NET Framework. Você aprenderá a projetar interfaces intuitivas e funcionais para seus aplicativos, aproveitando os recursos poderosos do Visual Studio para criar aplicativos com aparência profissional e funcionalidades ricas.

Ao longo desta apostila, você encontrará exemplos práticos, exercícios desafiadores e dicas úteis para ajudá-lo a consolidar o que aprendeu e aprimorar suas habilidades de programação.

1. Conhecendo a linguagem

1.1. O que é C#



A C# (pronuncia-se "C sharp") é uma linguagem de programação importante dentro do universo do .NET. Ela apareceu pela primeira vez quando o .NET foi lançado, lá em 2002.

Com C#, você pode criar todos os tipos de aplicativos: desde aqueles que ficam embutidos em dispositivos até aplicativos para computador, celulares e sites.

Por ser orientada a objetos, você pode construir blocos de código, chamados de classes e estruturas, que podem ser usados em diferentes partes do seu programa, economizando tempo e esforço.

E tem mais, a sintaxe dela é bem fácil de entender, o que torna a aprendizagem mais tranquila. Isso acontece porque ela pegou emprestado o que tinha de bom nas linguagens C e C++, além de algumas coisas avançadas do Java e outras linguagens por aí.

As características principais do C# são:

- **Suporte à orientação a objetos:** Com o C#, você pode criar classes, objetos e interfaces, e usar conceitos legais como herança, polimorfismo e encapsulamento.
- **Segurança de tipos:** Com o C#, quando você declara uma variável, você precisa dizer qual é o tipo dela. Depois disso, durante a execução, ela não pode mudar para outro tipo.
- **Coletor de lixo (Garbage Collector):** É uma ferramenta que cuida de limpar da memória automaticamente os objetos que não são mais usados pelo programa. Isso significa que você não precisa ficar se preocupando em liberar memória manualmente enquanto desenvolve com C#.
- **Tratamento de exceções:** O C# tem um sistema legal para pegar e lidar com erros que podem acontecer enquanto o programa está rodando.
- **Interoperabilidade:** Você pode usar o C# para interagir com programas feitos em outras linguagens, como C++, por exemplo. Isso é feito usando a CLR (Common Language Runtime), que permite usar bibliotecas de outras linguagens dentro do seu código C#.

1.2. Uma sintaxe simples

De forma parecida com outras linguagens de alto nível, num programa em C#, podemos dividir o código em diversos arquivos do tipo .cs — as conhecidas unidades de compilação.

Para criar esses arquivos, podemos usar um simples bloco de notas e o compilador csc.

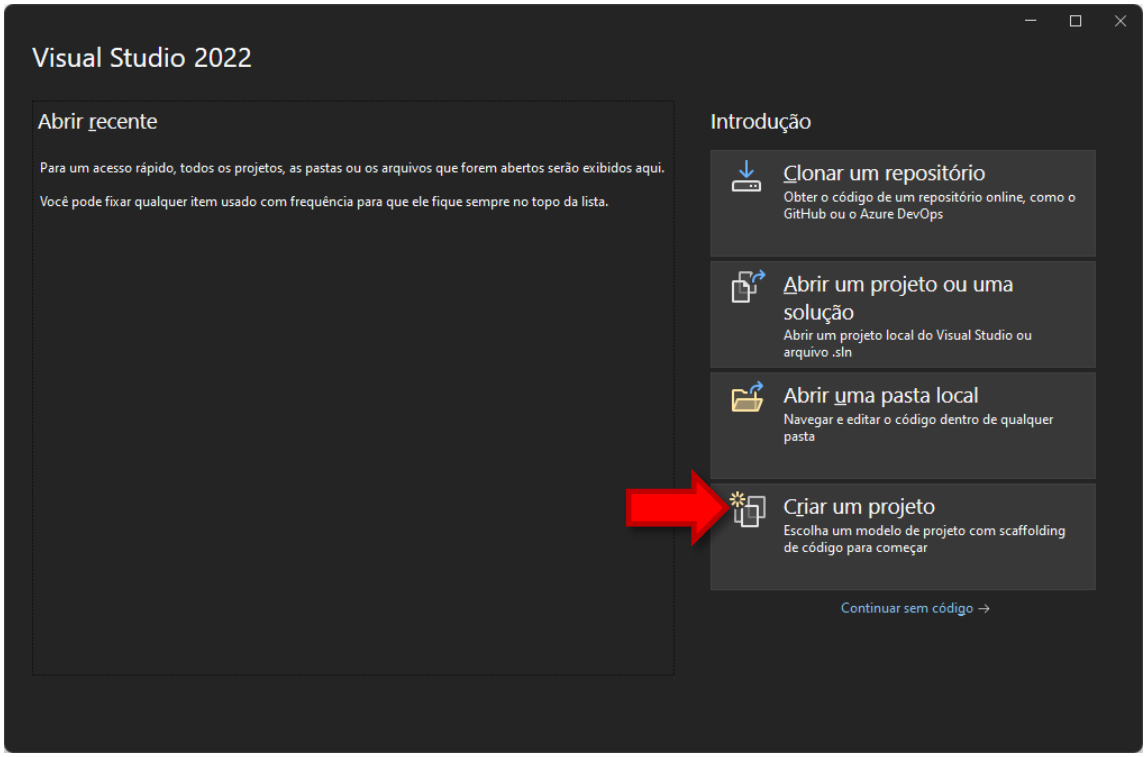
No entanto, o mais indicado é escolher uma ferramenta como Visual Studio Community, VS Code ou Rider (uma outra opção proprietária). No decorrer dessa apostila vamos utilizar o Microsoft Visual Studio Community na versão 2022.



1.2.1. Criando um projeto

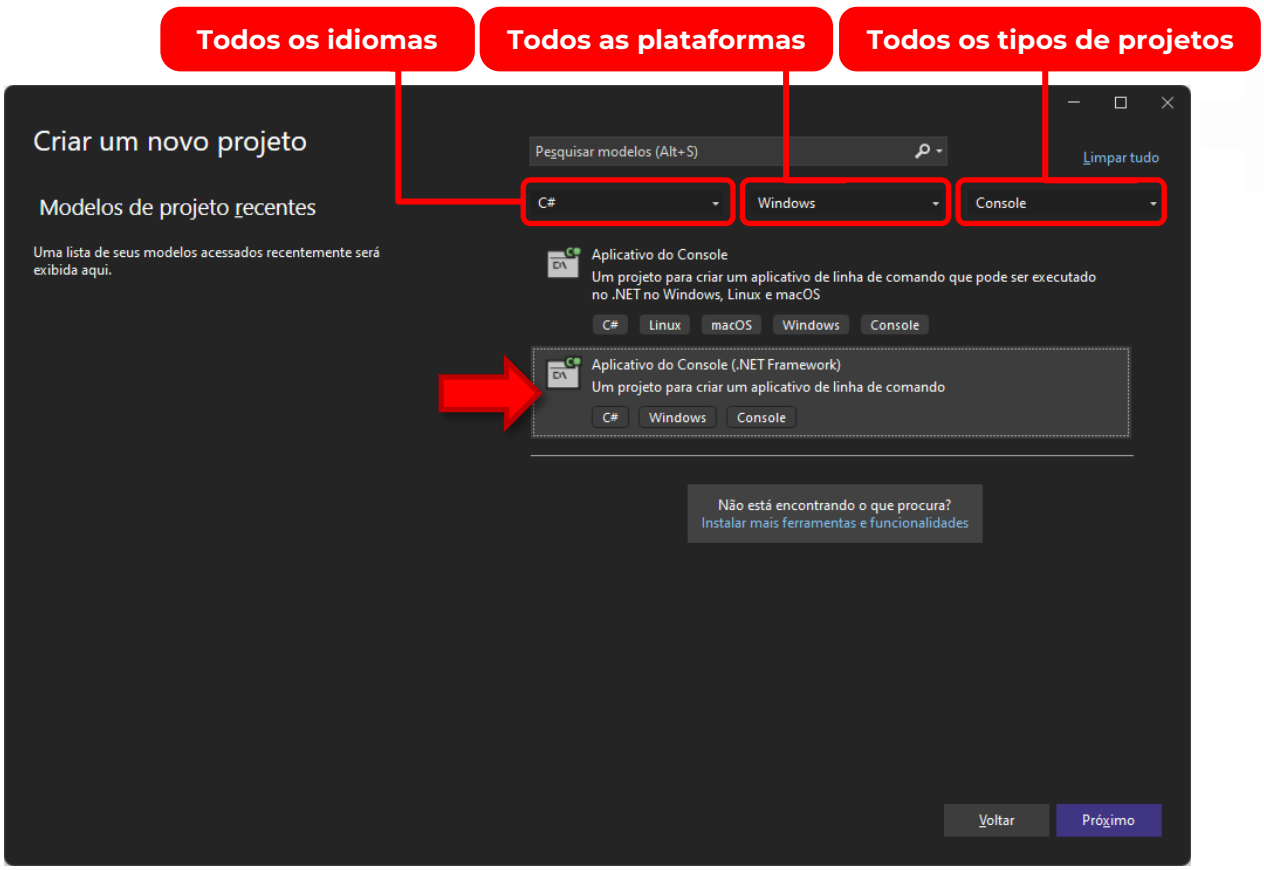
Para começar, crie um projeto de aplicativo em C#. O tipo de projeto vem com todos os arquivos de modelo que você precisa.

Abra o Visual Studio e selecione **Criar um projeto** na janela Iniciar.

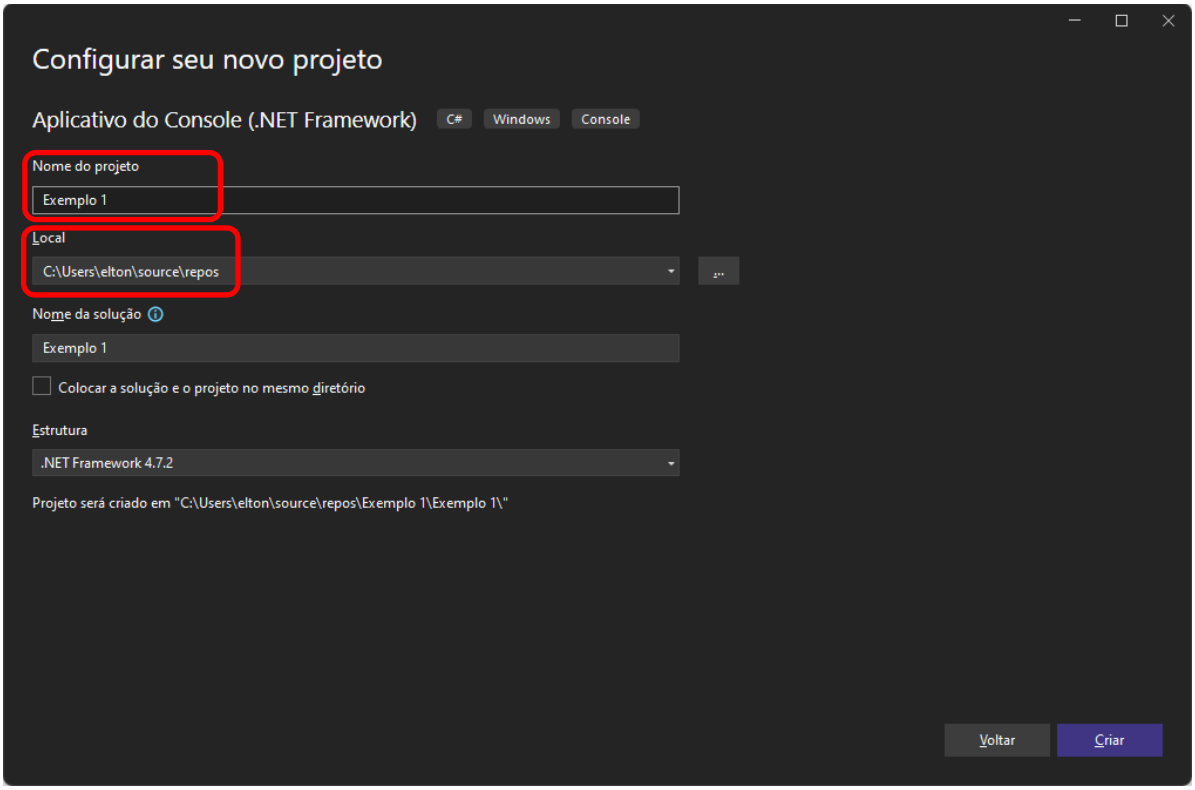


Na janela Criar um novo projeto, na lista suspensa **Todos os idiomas** escolha **C#**. Escolha **Windows** na lista **Todas as plataformas** e **Console** na lista **Todos os tipos de projeto**.

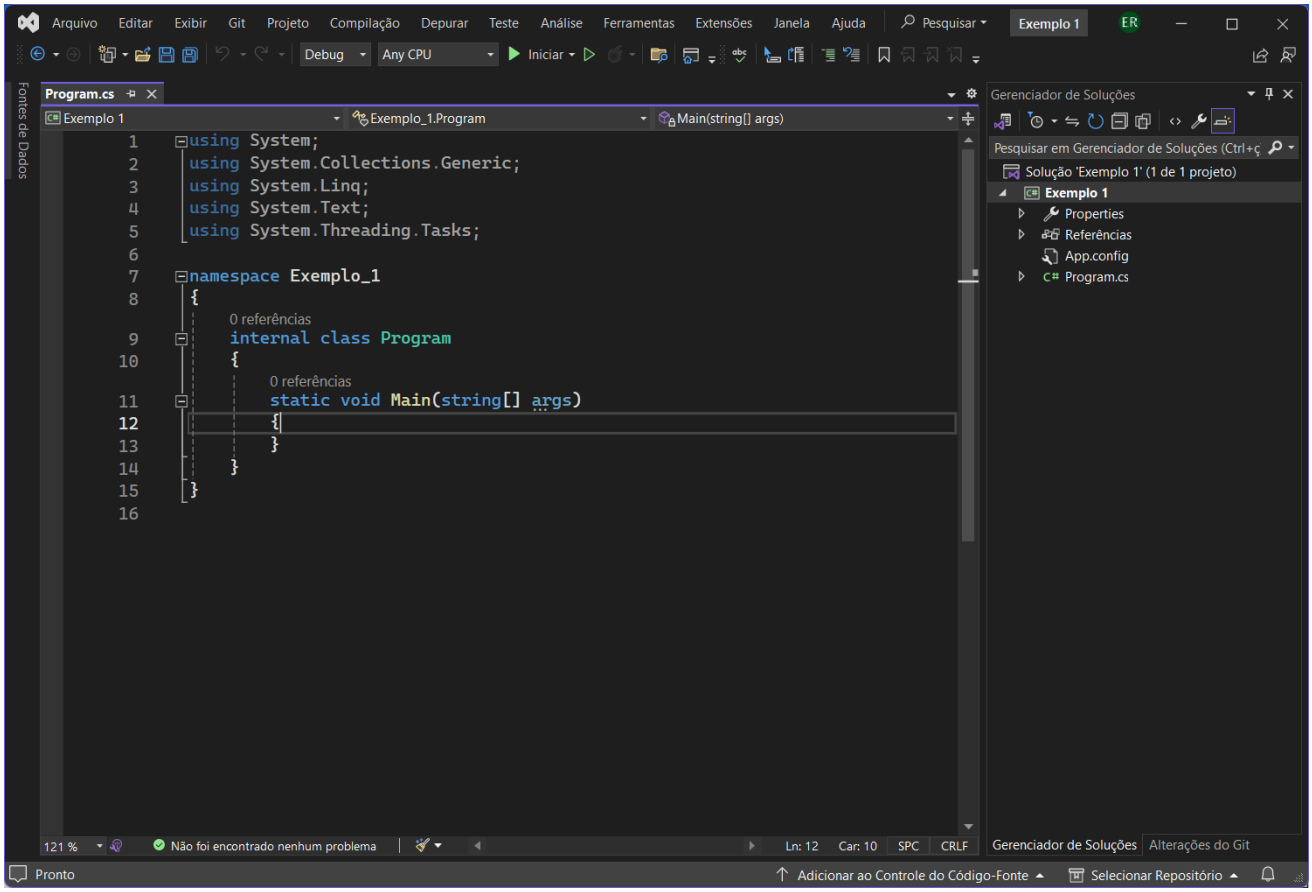
Depois de aplicar os filtros de idioma, plataforma e tipo de projeto, escolha o modelo **Aplicativo de Console (.NET Framework)** e, em seguida, selecione **Próximo**.



Na janela Configurar seu novo projeto, digite um nome de sua escolha na caixa **Nome do projeto**, escolha onde esse projeto será criado na caixa **Local** e, em seguida, selecione **Criar**.



O Visual Studio abre seu novo projeto, que inclui algumas linhas de código por padrão. Para exibi-lo no editor, selecione o arquivo de código Program.cs na janela **Gerenciador de Soluções**, que normalmente está no lado direito do Visual Studio.



Para fazer um simples “Olá mundo!!” podemos inserir um método `Writeline` para exibir a cadeia de caracteres. Veja o exemplo:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Exemplo_1
{
    0 referências
    internal class Program
    {
        0 referências
        static void Main(string[] args)
        {
            Console.WriteLine("Olá mundo!!");
        }
    }
}
```

Na linha 13 estamos usando o método `WriteLine` da classe `Console` que por sua vez fica localizada no Namespace `System` que simplesmente nos emite uma mensagem nesse caso '`Olá mundo!!`'. Atente-se que no final da instrução temos o ponto e virgula (`;`).

Para rodar essa aplicação devemos clicar no botão Iniciar na barra de ferramentas do Visual Studio ou simplesmente pressionar a tecla de atalho F5 no teclado.

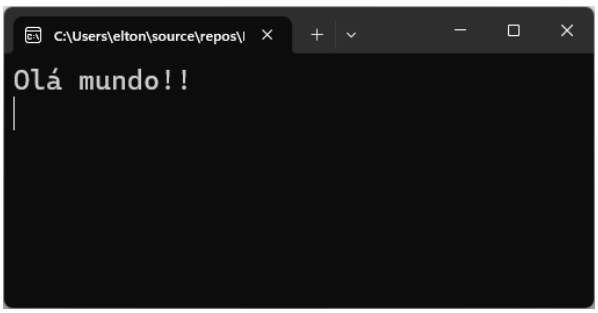
Quando você pressiona F5 no Visual Studio para rodar seu programa de C#, ele geralmente abre uma janelinha de terminal, mas fecha super rápido, certo? Isso acontece porque o programa roda tão rápido que mal dá tempo de ver alguma coisa.

Mas relaxa, tem um jeito de fazer essa janelinha ficar aberta até você dar uma olhada na mensagem que o programa mostra. Você pode usar o comando `Console.ReadKey()` no final do método `Main`. Isso faz com que o programa pare e espere até você pressionar qualquer tecla antes de fechar a janelinha. Assim, você tem tempo de ver a mensagem que o programa mostrou.

Então, seu código vai ficar assim:

```
1   using System;
2   using System.Collections.Generic;
3   using System.Linq;
4   using System.Text;
5   using System.Threading.Tasks;
6
7   namespace Exemplo_1
8   {
        0 referências
9       internal class Program
10      {
            0 referências
11          static void Main(string[] args)
12          {
13              Console.WriteLine("Olá mundo!!");
14              Console.ReadKey();
15          }
16      }
17  }
```

O terminal ficará aberto, exibindo a mensagem "Olá mundo!!" e aguardando ser fechado.



Vamos entender sua estrutura em partes:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

- **System:** Essa é uma caixinha mágica que contém muitas ferramentas úteis para fazer coisas legais em nossos programas. Coisas como imprimir mensagens na tela, trabalhar com textos, entre outras.
- **System.Collections.Generic:** Aqui temos um pacote com várias caixinhas extras que nos ajudam a organizar nossos dados de uma forma mais inteligente. São como armários especiais para guardar nossas coisas de forma organizada.
- **System.Linq:** Esse é um kit de ferramentas para fazer perguntas e buscar informações nos nossos dados. É como se fosse um detetive que nos ajuda a encontrar o que estamos procurando mais rapidamente.
- **System.Text:** Aqui temos mais ferramentas para lidar com textos e palavras. É como uma caixinha de truques para fazer coisas legais com letras e frases.
- **System.Threading.Tasks:** Essa é uma caixa que nos ajuda a fazer mais de uma coisa ao mesmo tempo no nosso programa. É útil quando precisamos realizar várias tarefas de uma vez, como enviar e-mails enquanto carregamos uma página da web.

```
namespace Exemplo_1
{
    internal class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

- **namespace Exemplo_1 { ... }:** Aqui estamos criando uma espécie de "quarto" especial para o nosso programa, onde podemos organizar tudo o que vamos usar.
- **internal class Program { ... }:** Aqui estamos definindo a "central" do nosso programa, que é onde tudo começa. É como a sala de controle onde vamos colocar o nosso código principal.
- **static void Main(string[] args) { ... }:** Este é o lugar onde tudo acontece! Quando o nosso programa é iniciado, ele começa a correr daqui. É como a porta de entrada para o nosso mundo digital.

2. Variáveis e tipos de dados

Na programação, os tipos de dados são usados para definir o tipo de valores que uma variável pode armazenar e as operações que podem ser realizadas sobre esses valores. Em C#, existem diversos tipos de dados, cada um com suas características e usos específicos.

2.1. Variáveis

Variáveis são como caixinhas ou potes onde você pode armazenar diferentes tipos de informações enquanto seu programa está rodando. Elas têm nomes, valores e tipos de dados associados a elas.

Vamos pensar em um exemplo simples: imagine que você está fazendo um jogo e precisa guardar a pontuação do jogador. Você pode usar uma variável para isso! Digamos que você chame essa variável de "pontuacao". Então, você pode dizer:

```
int pontuacao = 100;
```

Aqui, `int` é o tipo, `pontuacao` é o nome da variável, e `100` é o valor que estamos atribuindo a ela. Agora, sempre que precisar acessar ou modificar a pontuação do jogador no seu código, você pode simplesmente usar o nome da variável `pontuacao`.

Repare no "`;`" (ponto e vírgula) no final da linha. Como a declaração de uma variável é um comando da linguagem C#, precisamos do `;` para terminá-lo.

Depois de declarada, uma variável pode ser utilizada para armazenar valores. Por exemplo, se estivéssemos interessados em inserir o valor `200` na variável `pontuacao` que declaramos anteriormente, utilizaríamos o seguinte código:

```
pontuacao = 200;
```

Lê-se "pontuação recebe duzentos".

2.1.1. Regras para nomes de variáveis

A regra para nomear uma variável é que o nome dela sempre comece por uma **letra** ou _ (underline). No meio do nome podem-se usar números, mas não se devem usar caracteres especiais e também não pode ser uma palavra reservada. Entende-se por palavras reservadas os

comandos do C# e que são facilmente identificadas quando digitadas, por ficarem da cor azul. Exemplos de palavras que não podem ser utilizadas são: `if`, `for`, `while`, `string` e etc.

As variáveis em C# têm mais alguns detalhes importantes:

- Escolha nomes significativos e siga as convenções de nomenclatura, como nomeUsuario.
- Evite começar nomes de variáveis com números e evite caracteres especiais, exceto sublinhados.
- Lembre-se de que os nomes de variáveis são case-sensitive.
- Declare variáveis antes de utilizá-las no código.
- Prefira variáveis locais em vez de globais, sempre que possível.

2.2. Tipos de dados

Na programação, os tipos de dados são usados para definir o tipo de valores que uma variável pode armazenar e as operações que podem ser realizadas sobre esses valores.

C# é uma linguagem fortemente tipada, o que significa que cada variável deve ter um tipo definido e esse tipo não pode ser alterado depois da declaração. Nessa linguagem, existem diversos tipos de dados, cada um com suas características e usos específicos. Vamos explorar os principais tipos de dados em C#:

1. **Tipos de Dados Primitivos:**

- **Inteiros (Integer Types):** São usados para representar números inteiros. Existem vários tipos de inteiros em C#, como `int`, `short`, `long`, entre outros. O tipo `int` é o mais comum e geralmente tem um tamanho de 32 bits.

```
int idade = 25;
```

- **Pontos Flutuantes (Floating-Point Types):** São usados para representar números decimais. Os tipos de ponto flutuante em C# incluem `float` e `double`, sendo o `double` o mais comum. Eles são usados quando precisamos de valores precisos, como números com casas decimais.

```
float altura = 1.75f;
double pi = 3.14159265359;
```

- **Caracteres (Character Types):** O tipo `char` é usado para representar um único caractere Unicode de 16 bits. Cada caractere é representado por um número inteiro de 16 bits.
- **Booleanos (Boolean Type):** O tipo `bool` é usado para representar valores booleanos, ou seja, verdadeiro (true) ou falso (false). É amplamente usado em expressões lógicas e estruturas de controle de fluxo.

```
bool estaChovendo = true;
```

2. Tipos de Dados Compostos:

- **Cadeias de Caracteres (String Type):** O tipo `string` é usado para representar uma sequência de caracteres Unicode. É frequentemente usado para armazenar texto em variáveis.

```
string nome = "Maria";
```

- **Arrays:** Os `arrays` em C# são usados para armazenar coleções de elementos do mesmo tipo. Eles podem ser `arrays` unidimensionais, bidimensionais ou multidimensionais. Os `arrays` são úteis quando precisamos armazenar múltiplos valores em uma única variável.

3. Tipos de Dados Personalizados:

- **Enums:** Os enums, ou enumerações, são tipos de dados definidos pelo usuário que consistem em um conjunto de constantes nomeadas. Eles são úteis para representar conjuntos fixos de valores, como dias da semana ou opções de menu.
- **Structs:** As structs são tipos de dados semelhantes às classes, mas são tipos de valor em vez de tipos de referência. Elas são usadas para representar um grupo relacionado de campos que podem ser tratados como uma unidade.
- **Classes:** As classes são tipos de dados usados para definir objetos. Elas podem conter dados (campos) e operações (métodos) que manipulam esses dados. As classes são fundamentais para a programação orientada a objetos em C#.

4. Tipos de Dados Especiais:

- **Void:** O tipo void é usado para indicar que um método não retorna nenhum valor. Ele é usado em métodos que executam uma ação sem produzir um resultado.
- **Nullable Types:** Os tipos anuláveis são uma extensão dos tipos de valor que permitem que eles tenham um valor nulo adicional, além dos valores normais do tipo. Isso é útil quando precisamos representar a ausência de um valor, além de valores reais.

3. Programando no Console .NET

O Console .NET é uma ferramenta que nos permite interagir com nossos programas diretamente no terminal do computador. É como se fosse uma conversa entre a gente e o computador, onde podemos pedir informações e ele nos responde!

3.1. Escrevendo e Lendo Dados no Console .NET

Vamos começar com algo simples. Imagine que estamos criando um programa para saber o nome dos nossos amigos. Precisamos que eles nos digam seus nomes, né? Então, vamos aprender como fazer isso!

Para ler os nomes que nossos amigos digitam, usamos o método `Console.ReadLine()`.

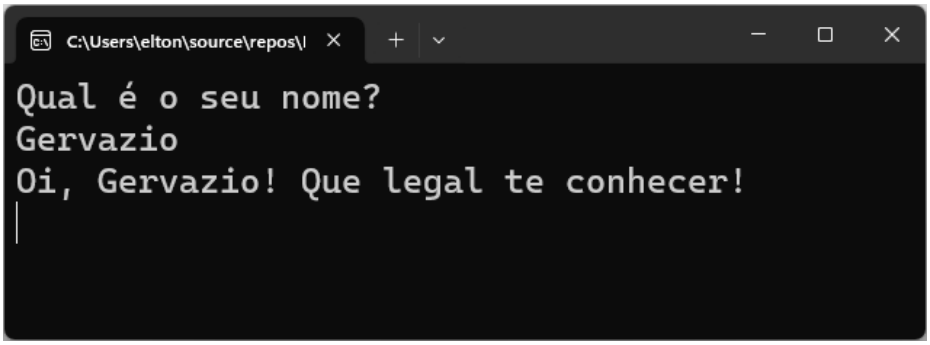
```
string nomeAmigo;
Console.WriteLine("Qual é o seu nome?");
nomeAmigo = Console.ReadLine();
Console.WriteLine("Oi, " + nomeAmigo + "! Que legal te conhecer!");
```

Nesse exemplo usamos o operador de concatenação `+` para juntar várias strings em uma só antes de exibi-las na tela. Aqui, estamos usando três strings diferentes:

- A primeira é a string literal `"Oi, "`, que é uma mensagem fixa que queremos mostrar.
- A segunda é a variável `nomeAmigo`, que contém o nome que o usuário digitou.
- A terceira é a string literal `"! Que legal te conhecer!"`, outra mensagem fixa.

O que acontece é que o C# entende que queremos unir essas strings e exibir o resultado como uma mensagem completa.

Então, quando o programa rodar, ele vai mostrar algo como: **"Oi, [nome do amigo]! Que legal te conhecer!"**, onde **[nome do amigo]** é o nome que o usuário digitou.



3.1.1. Conversão de dados

Quando usamos `Console.ReadLine()`, o que recebemos é uma string, ou seja, um texto. Mas e se precisarmos usar essas informações como números? Para isso, precisamos converter a string para o tipo de dado desejado.

Vamos ver como fazer isso com um exemplo para somar dois números que o usuário digita. Dessa vez, vamos precisar converter os números de texto para números de verdade!

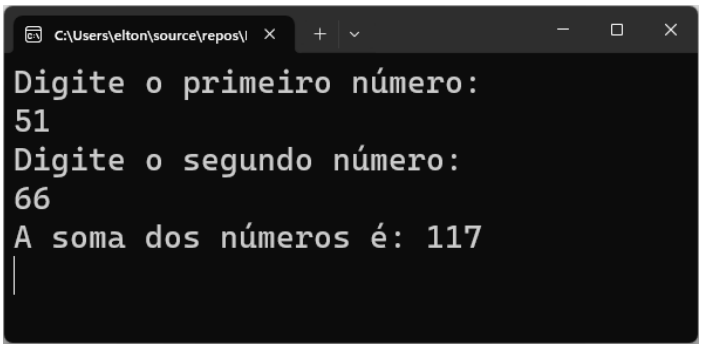
```
string inputNumero1, inputNumero2;
int numero1, numero2, resultado;

Console.WriteLine("Digite o primeiro número:");
inputNumero1 = Console.ReadLine();
numero1 = int.Parse(inputNumero1);

Console.WriteLine("Digite o segundo número:");
inputNumero2 = Console.ReadLine();
numero2 = int.Parse(inputNumero2);

resultado = numero1 + numero2;
Console.WriteLine("A soma dos números é: " + resultado);
```

Neste exemplo, a gente lê os números como texto usando `Console.ReadLine()`, depois a gente converte eles para inteiros usando `int.Parse()`, e finalmente a gente soma os números e mostra o resultado.



É possível fazer o mesmo reduzindo um pouco o tamanho do nosso código. Veja o exemplo:

```
numero1 = int.Parse(Console.ReadLine());
```

Neste exemplo, a gente lê os números como texto usando `Console.ReadLine()`, envolvendo-o dentro da função de conversão para inteiros `int.Parse()`, dispensando o uso das variáveis `inputNumero1` e `inputNumero2`.

```
int numero1, numero2, resultado;

Console.WriteLine("Digite o primeiro número:");
numero1 = int.Parse(Console.ReadLine());

Console.WriteLine("Digite o segundo número:");
numero2 = int.Parse(Console.ReadLine());

resultado = numero1 + numero2;
Console.WriteLine("A soma dos números é: " + resultado);
```

Você estará fazendo o mesmo que no exemplo anterior, porém de uma forma mais compacta.

Essa linha de código faz o seguinte:

1. Utiliza `Console.ReadLine()` para ler uma linha de texto digitada pelo usuário no console.
2. Usa `int.Parse()` para converter esse texto em um número inteiro (`int`) e armazená-lo na variável `numero1`.

Essa forma é bastante comum em C# e é uma maneira mais direta de realizar a conversão de uma string para um número inteiro.

4. Estruturas de controle

Quem nunca ouviu falar daquele if/else que os programadores comentam? Essas duas palavrinhas são parte da estrutura de controle que temos nos nossos sistemas. Elas são necessárias quando o programa precisa tomar uma determinada decisão mediante o valor de algum cálculo ou uma varíavel e, dependendo de seu valor, o caminho pode ser x ou y durante a execução do software.

4.1. Tomando decisões no código

Vamos usar um exemplo de uma aplicação bancária, queremos permitir um saque somente se o valor a ser retirado for menor ou igual ao saldo da conta, ou seja, se o saldo da conta for maior ou igual ao valor do saque, devemos permitir a operação, do contrário não podemos permitir o saque. Precisamos fazer execução condicional de código.

No C#, podemos executar código condicional utilizando a construção if:

```
if (condicao)
{
    // Esse código será executado somente se a condição for verdadeira
}
```

No nosso exemplo, queremos executar a lógica de saque apenas se o saldo for maior ou igual ao valor do saque:

```
double saldo = 100.0;
double valorSaque = 10.0;

if (saldo >= valorSaque)
{
    // código do saque.
}
```

O código do saque deve diminuir o saldo da conta e mostrar uma mensagem para o usuário indicando que o saque ocorreu com sucesso:

```
double saldo = 100.0;
double valorSaque = 10.0;

if (saldo >= valorSaque)
{
    saldo = saldo - valorSaque;
    Console.WriteLine("Saque realizado com sucesso");
}
```

Repare que, se a conta não tiver saldo suficiente para o saque, o usuário não é avisado. Então estamos na seguinte situação: "Se a conta tiver saldo suficiente, quero fazer o saque, senão, quero mostrar a mensagem Saldo Insuficiente para o usuário". Para fazer isso, podemos usar o else do C#:

```
double saldo = 100.0;
double valorSaque = 10.0;

if (saldo >= valorSaque)
{
    saldo = saldo - valorSaque;
    Console.WriteLine("Saque realizado com sucesso");
}

else
{
    Console.WriteLine("Saldo Insuficiente");
}
```

4.2. Mais sobre condições

Repare na expressão que passamos para o `if: saldo >= valorSaque`. Nele, utilizamos o operador "maior ou igual". Além dele, existem outros operadores de comparação que podemos utilizar: maior (`>`), menor (`<`), menor ou igual (`<=`), igual (`==`) e diferente (`!=`). Podemos também negar uma condição de um `if` utilizando o operador `!` na frente da condição que será negada.

Também podemos realizar algumas operações com valores do tipo `bool`. Podemos, por exemplo, verificar se duas condições são verdadeiras ao mesmo tempo usando o operador `&&` (AND) para fazer um e lógico:

```
bool realmentePodeSacar = (saldo >= valorSaque) && (valorSaque > 0);
```

Quando precisamos de um **OU** lógico, utilizamos o operador `||`:

```
// essa condição é verdadeira se (saldo >= valorSaque) for true
// ou se (valorSaque > 0) for verdadeiro.

bool realmentePodeSacar = (saldo >= valorSaque) || (valorSaque > 0);
```

Assim, podemos construir condições mais complexas para um `if`. Por exemplo, podemos usar a variável `realmentePodeSacar` declarada no `if` que verifica se o cliente pode sacar ou não:

```
if (realmentePodeSacar)
{
    // código do saque
}
else
{
    Console.WriteLine("Saldo Insuficiente");
}
```

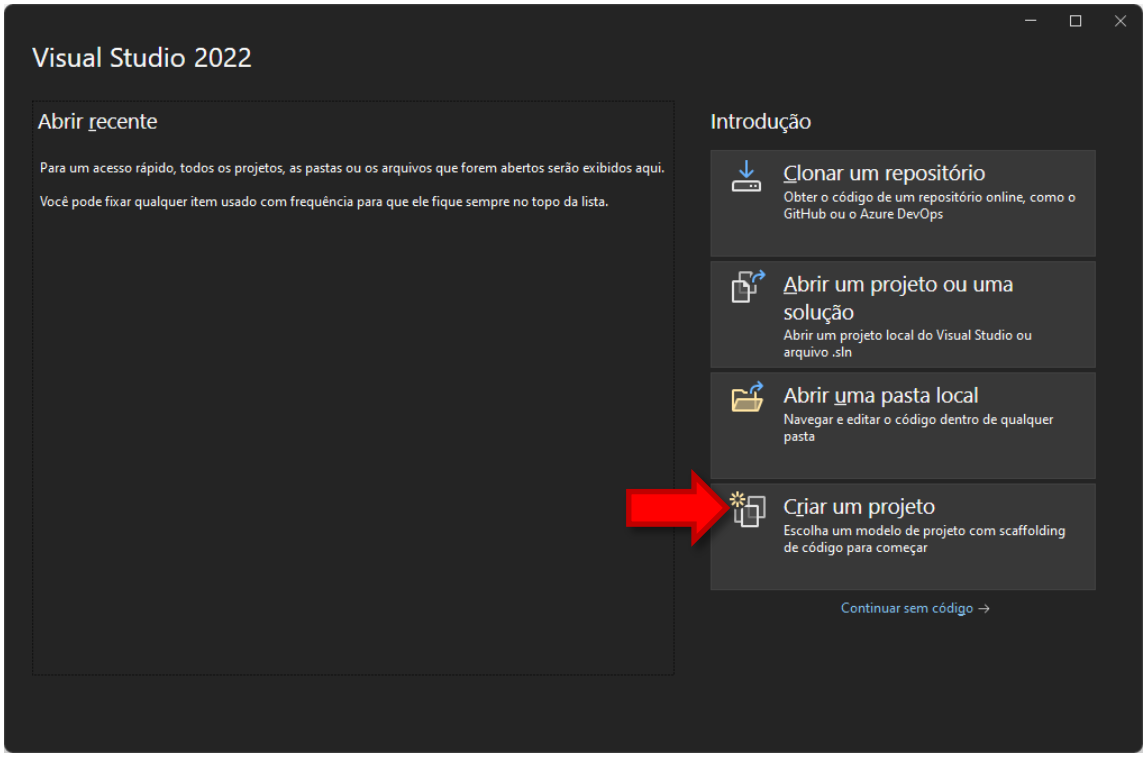
5. Windows Forms

Windows Forms, também conhecido como **WinForms**, é uma ferramenta que vem junto com o **.Net Framework** e ajuda a criar interfaces gráficas para programas de computador, como aqueles usados em desktops, tablets e PCs. Basicamente, é uma maneira mais fácil de fazer aplicativos bonitinhos para o seu computador.

Os programas feitos com Windows Forms são chamados de aplicativos Windows Forms, e eles funcionam no seu computador, não na internet. Com o WinForms, você pode adicionar diferentes coisinhas nas suas telas, tipo caixas de texto, rótulos e outras coisinhas legais para deixar seu programa mais bonito.

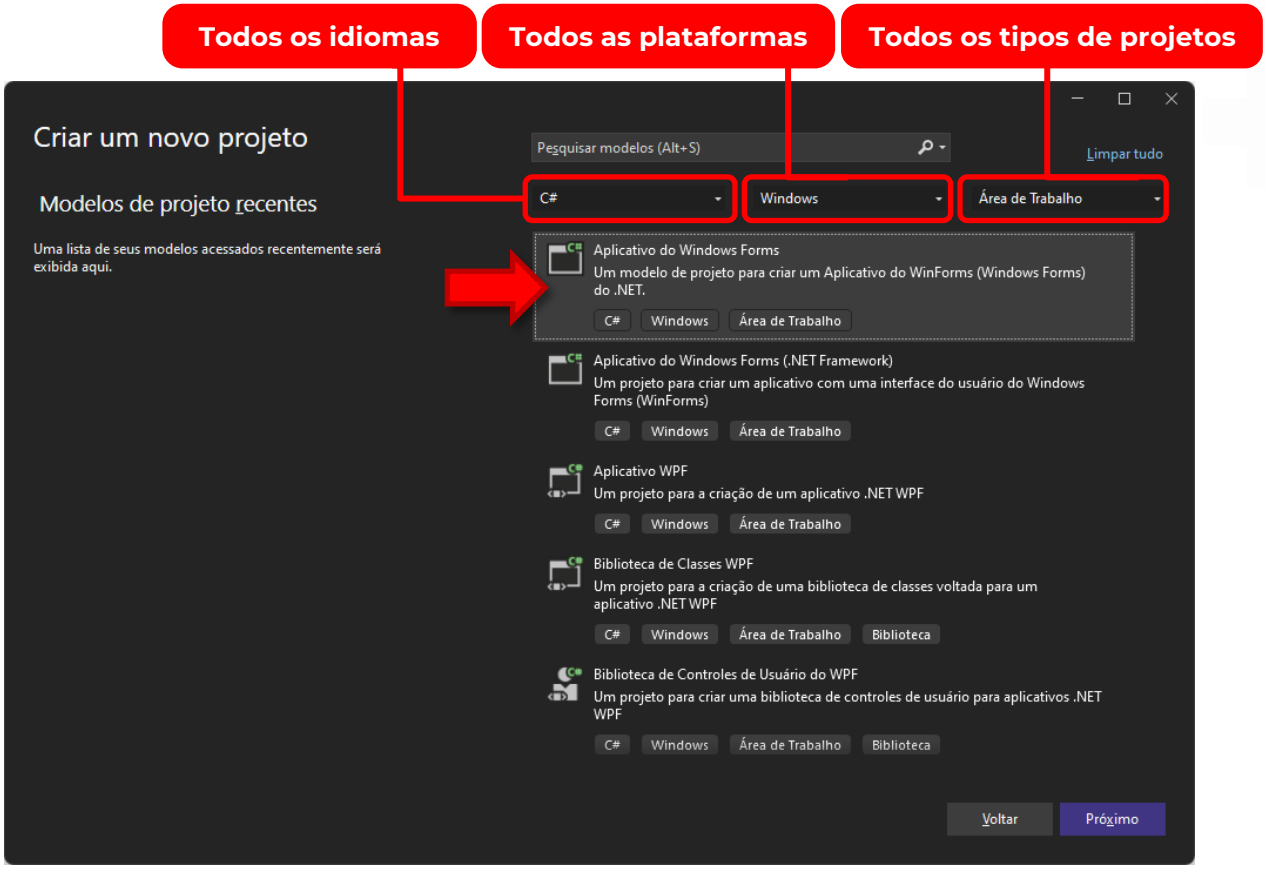
5.1. Criando um aplicativo Windows Forms

Abra o Visual Studio e selecione **Criar um projeto** na janela Iniciar.

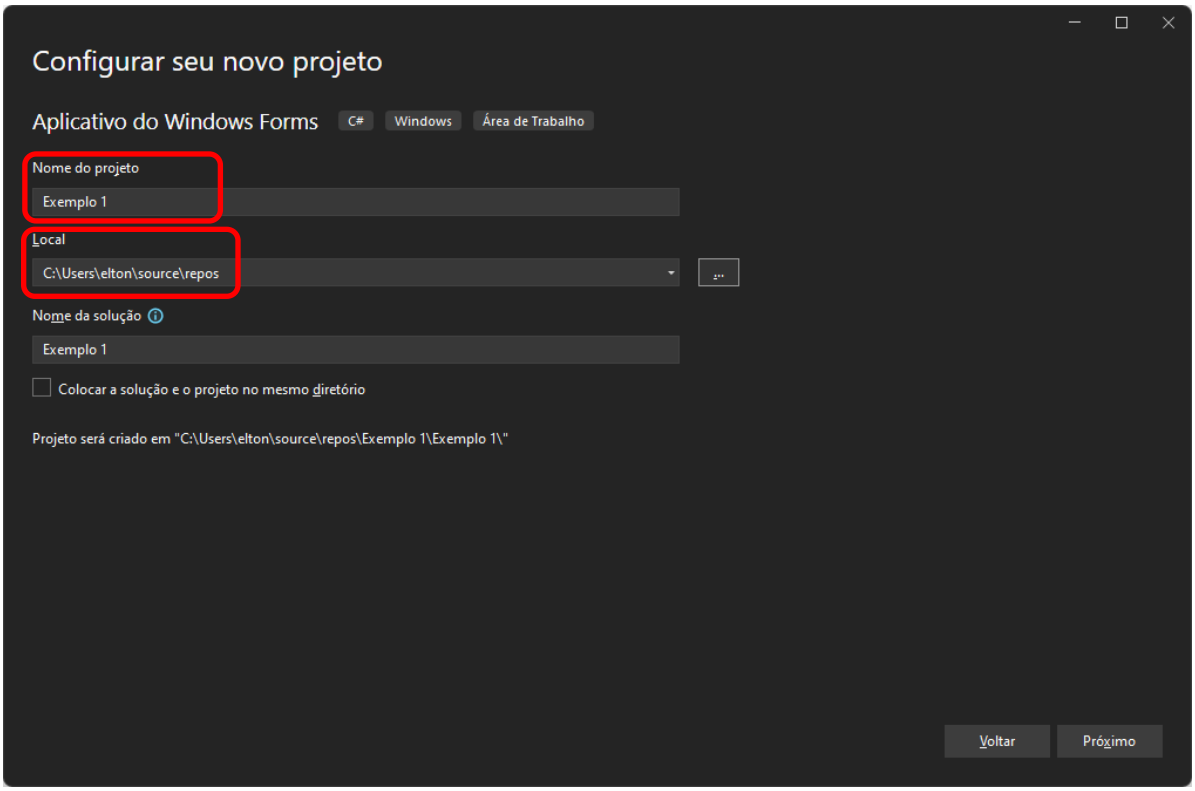


Na janela Criar um novo projeto, na lista suspensa **Todos os idiomas** escolha **C#**. Escolha **Windows** na lista **Todas as plataformas** e **Área de Trabalho** na lista **Todos os tipos de projeto**.

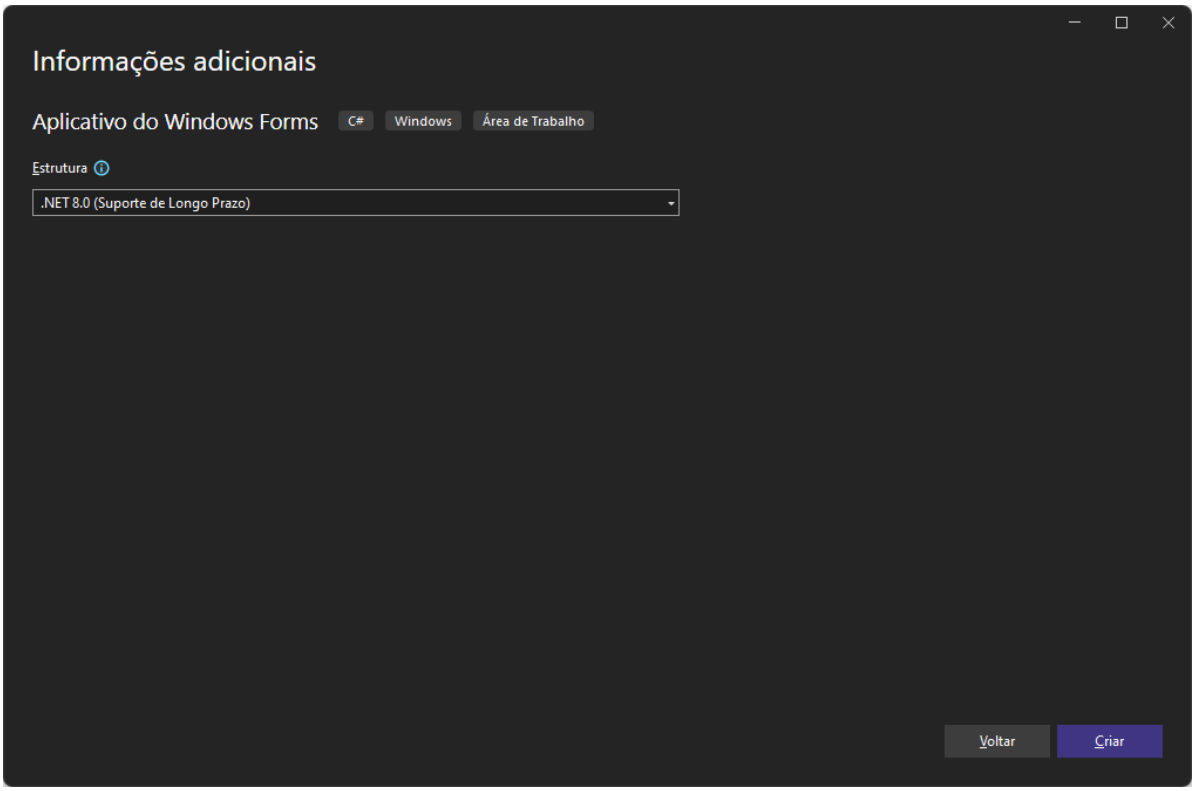
Depois de aplicar os filtros de idioma, plataforma e tipo de projeto, escolha o modelo **Aplicativo do Windows Forms** e, em seguida, selecione **Próximo**.



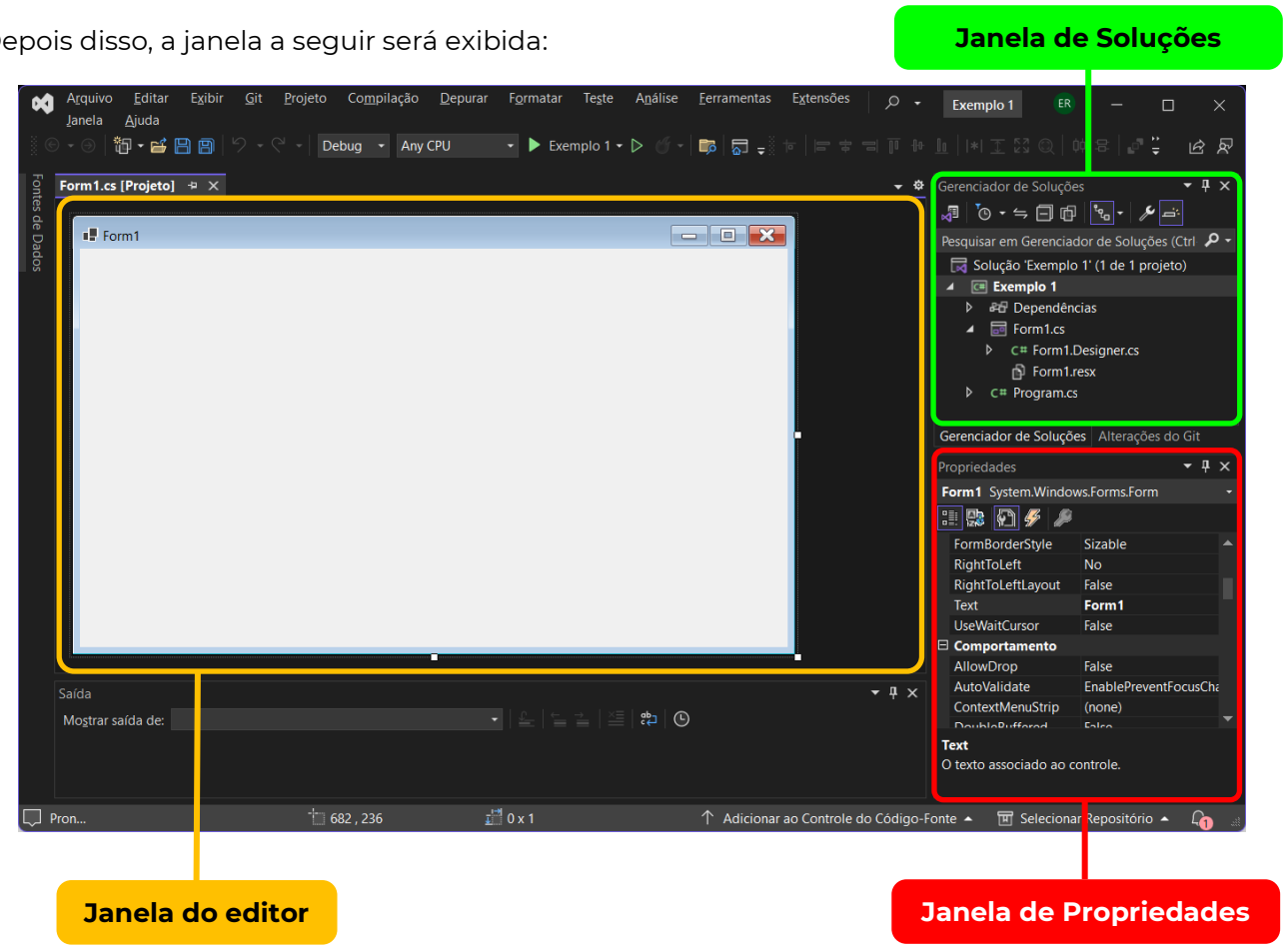
Na janela Configurar seu novo projeto, digite um nome de sua escolha na caixa **Nome do projeto**, escolha onde esse projeto será criado na caixa **Local** e, em seguida, selecione **Criar**.



Na janela Informações adicionais, mantenha a opção **.NET 8.0 (Suporte de Longo Prazo)** no campo **Estrutura** e depois clique em **Criar**



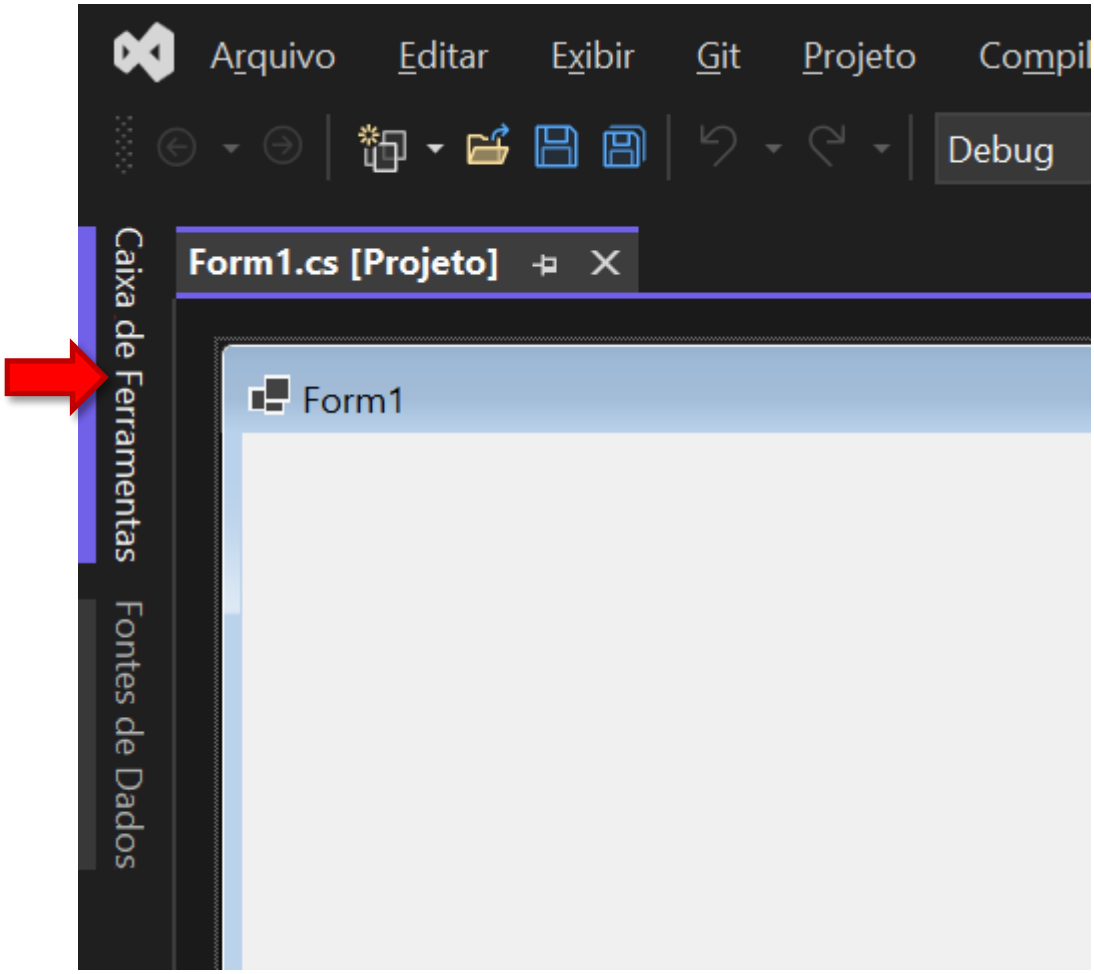
Depois disso, a janela a seguir será exibida:



1. **Janela do editor ou janela principal:** aqui, você trabalhará com formulários e edição de código. Você pode notar o layout do formulário que agora está em branco. Você irá clicar duas vezes no formulário e então ele abrirá o código para isso.
2. **Janela ou Gerenciador de Soluções:** é usada para navegar entre todos os itens da solução. Por exemplo, se você selecionar um arquivo desta janela, informações específicas serão exibidas na janela de propriedades.
3. **Janela de propriedades:** esta janela é usada para alterar as diferentes propriedades do item selecionado no Gerenciador de Soluções. Além disso, você pode alterar as propriedades dos componentes ou controles que adicionará aos formulários.

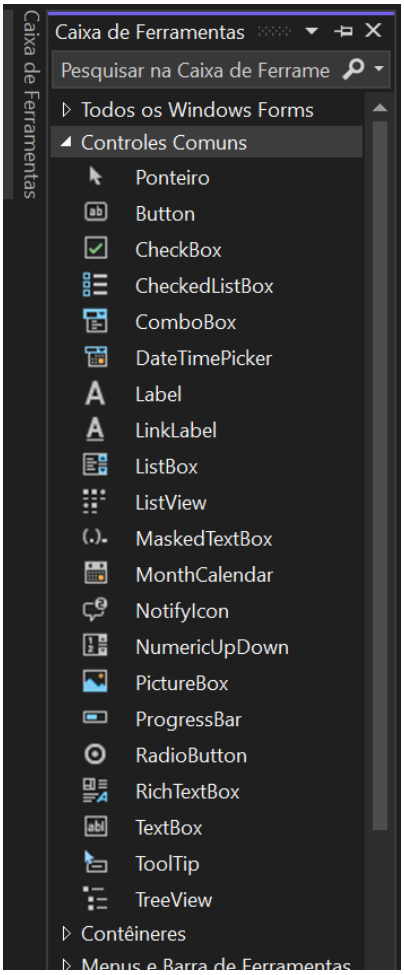
Você também pode redefinir o layout da janela configurando-o para o padrão. Para definir o layout padrão, vá para **Janela -> Redefinir Layout da Janela** no Menu do Visual Studio.

Agora, para adicionar os controles ao seu aplicativo Windows Forms, vá para a guia **Caixa de Ferramentas** presente na extremidade esquerda do Visual Studio.



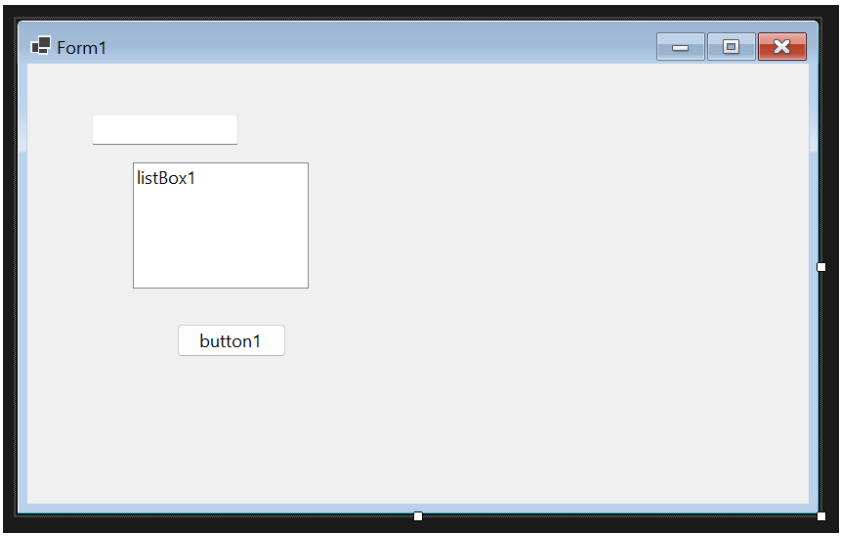
Caso essa guia não esteja sendo exibida vá no menu **Exibir -> Caixa de Ferramentas**.

5.2. Adicionando Controles ao Formulário

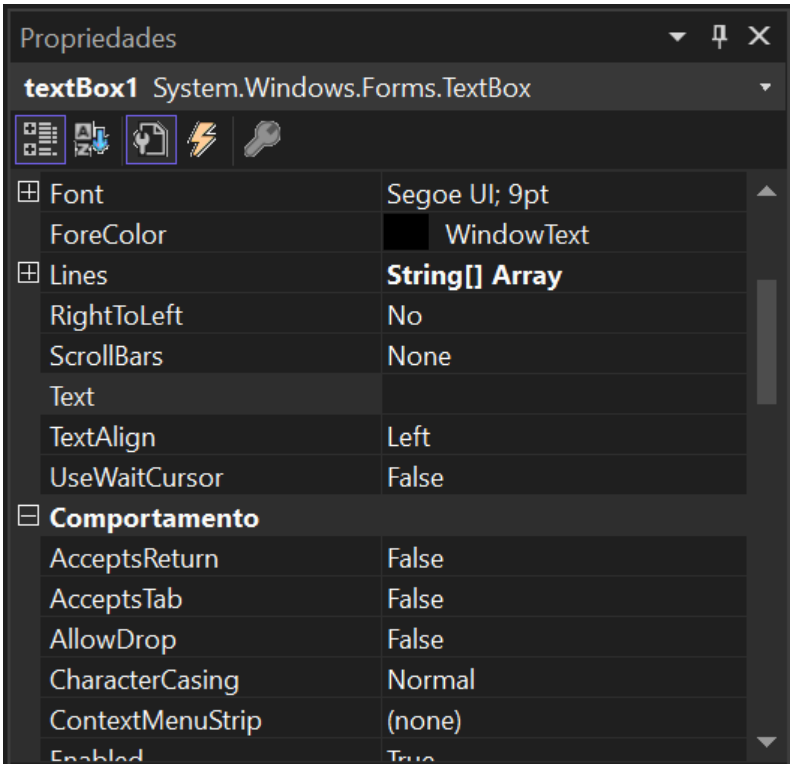


Aqui, você pode ver uma lista de controles. Para acessar os controles mais comumente usados, vá para **Controles Comuns** presentes na guia Caixa de Ferramentas.

Agora arraste e solte os controles necessários no Formulário criado. Por exemplo, pode adicionar **TextBox**, **ListBox**, **Button** etc. conforme mostrado abaixo.



Ao clicar no controle solto específico, você pode ver e alterar suas propriedades presentes no canto direito do Visual Studio.



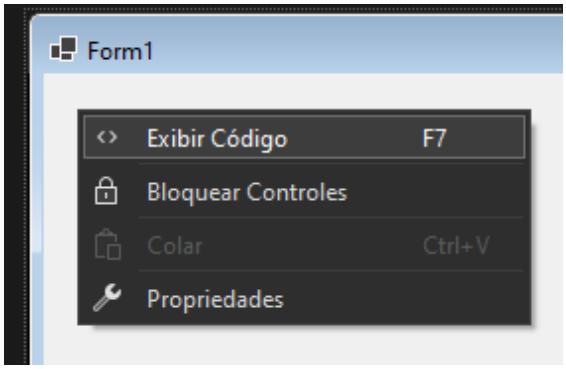
Se o controle TextBox estiver selecionado, você verá suas propriedades como **TextAlign**, **MaxLength** etc. Você pode alterar os valores de suas propriedades de acordo com a necessidade do aplicativo.

Para excluir os controles textBox, listBox e button basta selecioná-los clicando com o botão direito do mouse sobre eles e clicar na opção **Excluir** ou pressionando a tecla **Delete** no teclado.

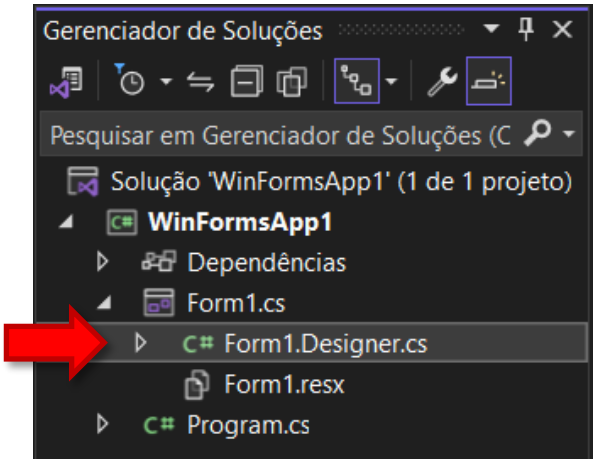
5.3. Explorando os Arquivos .cs e .Designer.cs

Quando você cria um formulário no Visual Studio, dois arquivos são gerados para cada formulário:

- **Form1.cs:** Este arquivo contém a classe principal do formulário, onde você pode adicionar código para manipular eventos e interações. Para visualizá-lo basta clicar com o botão direito do mouse sobre o formulário na Janela do Editor, escolher a opção **Exibir Código** ou pressionar a tecla F7 no teclado.



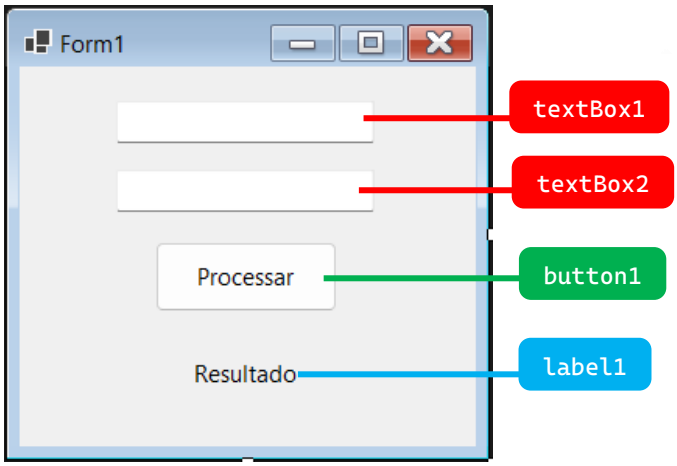
- **Form1.Designer.cs:** Este arquivo contém o código gerado automaticamente pelo Visual Studio, incluindo a inicialização dos controles e sua posição no formulário. Você normalmente não precisa editar este arquivo manualmente. Para acessá-lo basta clicar sobre ele na Janela de Soluções.



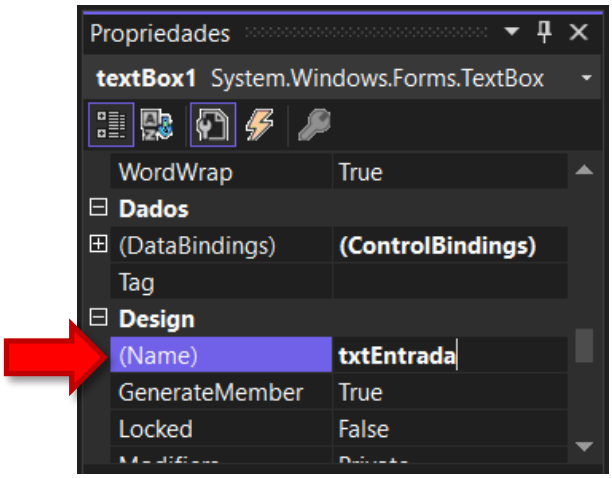
5.3.1. Atribuindo Nomes aos Controles

Para facilitar a referência aos controles no código, é uma boa prática atribuir nomes a eles. Adicione os seguintes controles ao formulário:

- Dois TextBox (textBoxEntrada e textBoxSaida)
- Um Botão (btnProcessar)
- Uma Label (labelResultado)



Você pode atribuir nomes à eles usando a Janela de Propriedades. Selecione um controle, como o TextBox textBoxEntrada. Na janela de propriedades, localize a propriedade "Name". Note que, por padrão, esses controles vêm com os nomes: `textBox1`, `textBox2`, `button1` e `label1`. Agora atribua um nome significativo, como "**txtEntrada**" para o `textBox1`.

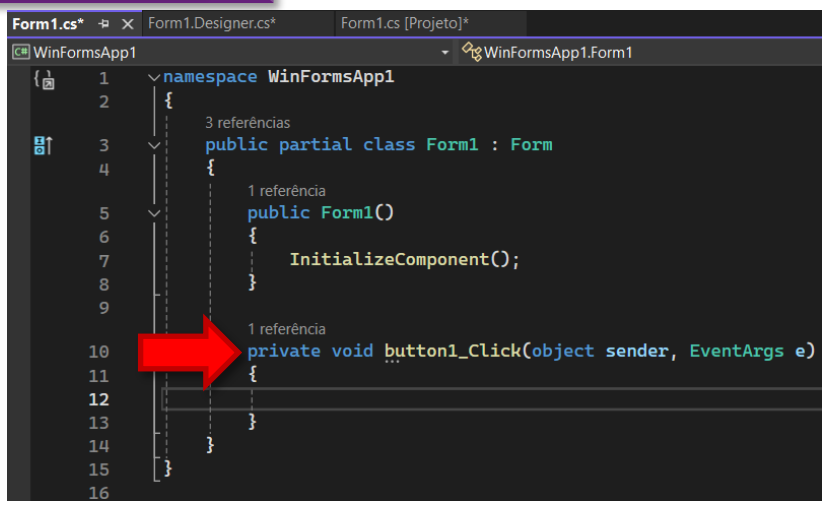


Faça o mesmo para os outros controles: "**txtSaida**", "**btnProcessar**" e "**lblResultado**".

5.3.2. Atribuindo Comandos ao Botão

Agora, vamos atribuir um comando ao Botão para que, ao ser clicado, ele pegue o texto do TextBox textBoxEntrada e exiba no TextBox textBoxSaida e na Label labelResultado.

Dê um duplo clique no Botão (btnProcessar) para criar um evento de clique. Veja que o arquivo Form1.cs é carregado e um evento chamado **btnProcessar_Click** é criado automaticamente.



Adicione o seguinte código ao 'btnProcessar_Click':

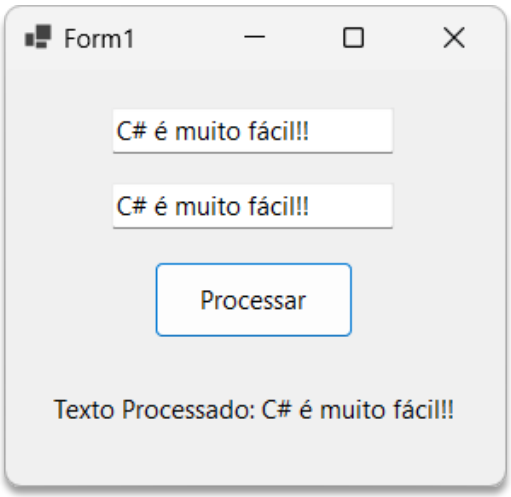
```
private void btnProcessar_Click(object sender, EventArgs e)
{
    string textoEntrada = txtEntrada.Text;

    txtSaida.Text = textoEntrada;

    lblResultado.Text = "Texto Processado: " + textoEntrada;
}
```

Agora que tudo está configurado, você pode testar a aplicação. Insira algum texto no textBoxEntrada depois clique no Botão (btnProcessar).

O texto inserido deve aparecer no textBoxSaida e na labelResultado.



Veja que o arquivo **Form1.Designer.cs** exibe a inicialização dos controles e suas posições dentro do formulário.

```
1 referência
private void InitializeComponent()
{
    txtEntrada = new TextBox();
    txtSaida = new TextBox();
    btnProcessar = new Button();
    lblResultado = new Label();
    SuspendLayout();
    //
    // txtEntrada
    //
    txtEntrada.Location = new Point(62, 22);
    txtEntrada.Name = "txtEntrada";
    txtEntrada.Size = new Size(166, 27);
    txtEntrada.TabIndex = 0;
    //
    // txtSaida
    //
    txtSaida.Location = new Point(62, 66);
    txtSaida.Name = "txtSaida";
    txtSaida.Size = new Size(166, 27);
    txtSaida.TabIndex = 1;
    //
    // btnProcessar
    //
    btnProcessar.Location = new Point(87, 112);
    btnProcessar.Name = "btnProcessar";
    btnProcessar.Size = new Size(117, 45);
    btnProcessar.TabIndex = 2;
    btnProcessar.Text = "Processar";
    btnProcessar.UseVisualStyleBackColor = true;
    btnProcessar.Click += btnProcessar_Click;
    //
    // lblResultado
    //
```

5.4. Convertendo o Valor do TextBox em uma Variável

Ao trabalhar com dados inseridos pelo usuário em uma aplicação Windows Forms, é comum precisarmos converter esses dados de texto para outros tipos de dados, como números inteiros, decimais, datas, entre outros. Vamos ver como fazer isso e armazenar o resultado em uma variável.

Utilizando o mesmo exemplo anterior, vamos apenas alterar o código para que os valores de cada `textBox` sejam somados e exibidos na `label`:

```
private void btnProcessar_Click(object sender, EventArgs e)
{
    int textoEntrada = int.Parse(txtEntrada.Text);
    int textoSaida = int.Parse(txtEntrada.Text);

    int resultado = textoEntrada + textoSaida;

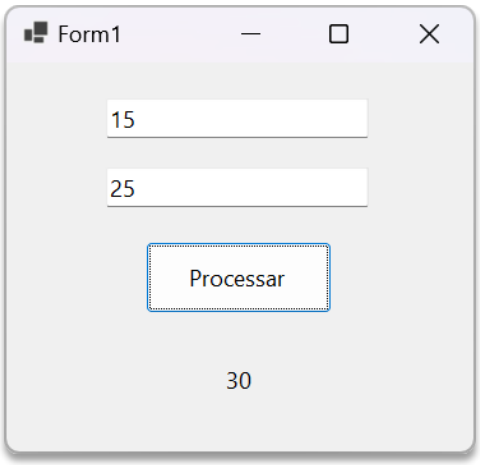
    lblResultado.Text = resultado.ToString();
}
```

A função `int.Parse` converte uma string em um número inteiro. Estamos obtendo o texto de txtEntrada usando txtEntrada.Text e convertendo para um inteiro. O resultado é armazenado nas variáveis `textoEntrada` e `textoSaida`.

Em seguida, realizamos uma soma com os valores contidos nas variáveis `textoEntrada` e `textoSaida` e armazenando o resultado na variável `resultado`.

Repare que foi atribuída a label `lblResultado` a variável resultado juntamente com o método `ToString()`. Esse método é usado para converter o valor de resultado de inteiro para string. Como a propriedade Text de lblResultado espera uma string, usamos `ToString()` para garantir que estamos atribuindo um valor do tipo string à propriedade.

Veja em execução:



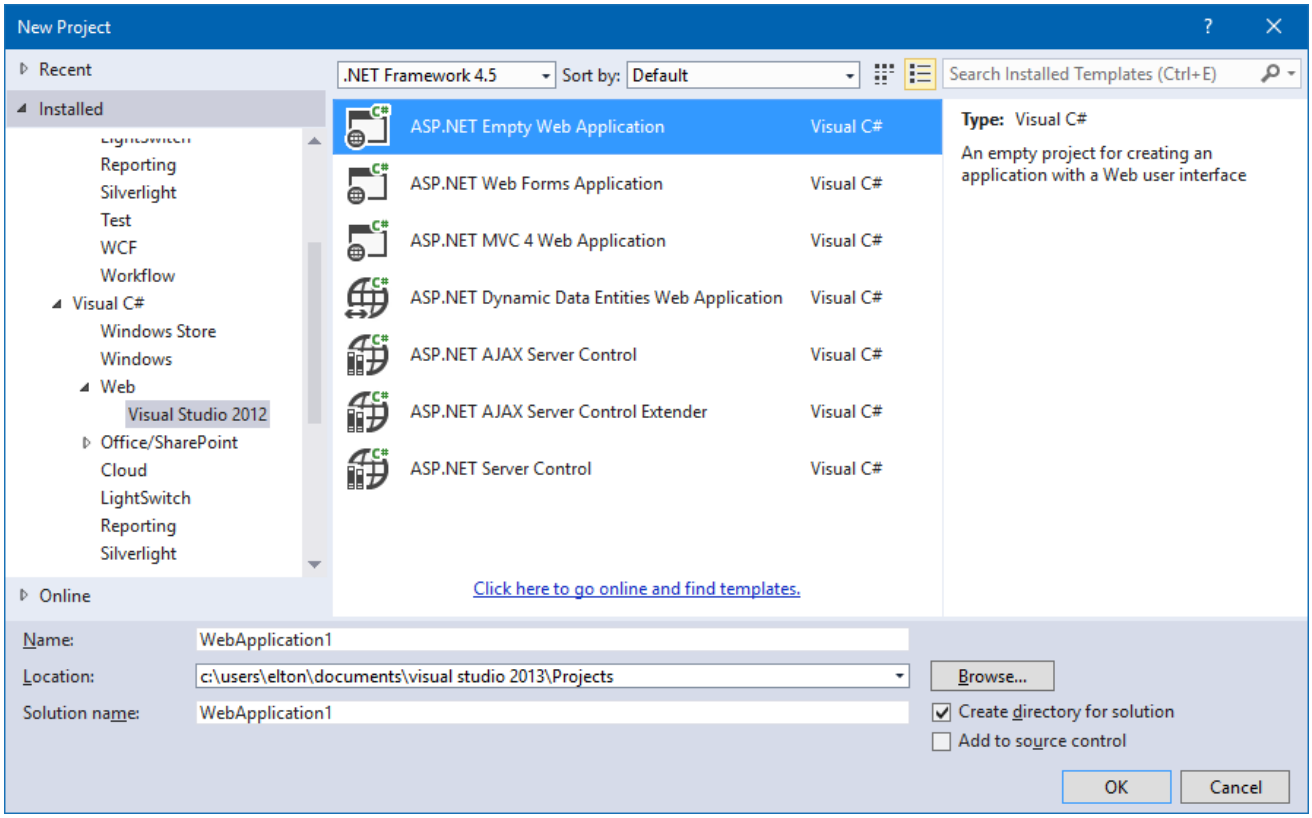
6. Banco de Dados

É possível exibir rápida e facilmente dados com base em um arquivo de banco de dados local no aplicativo criando-se um conjunto de dados e, em seguida, adicionando-se controles de associação de dados ao aplicativo.

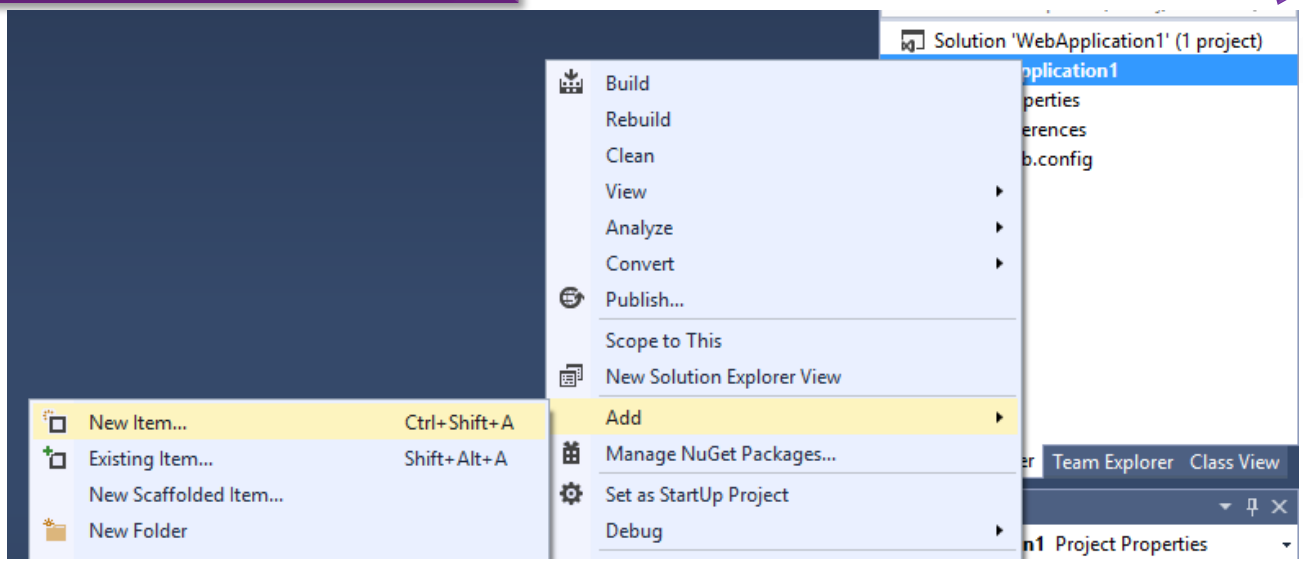
6.1. Criando e configurando um conjunto Banco de Dados

Durante a criação de um banco de dados no Visual Studio 2013, o mecanismo LocalDB do SQL Server Express é usado para acessar um arquivo de banco de dados (.mdf) no SQL Server 2012. Em versões anteriores do Visual Studio, o mecanismo do SQL Server Express é usado para acessar um arquivo de banco de dados (.mdf).

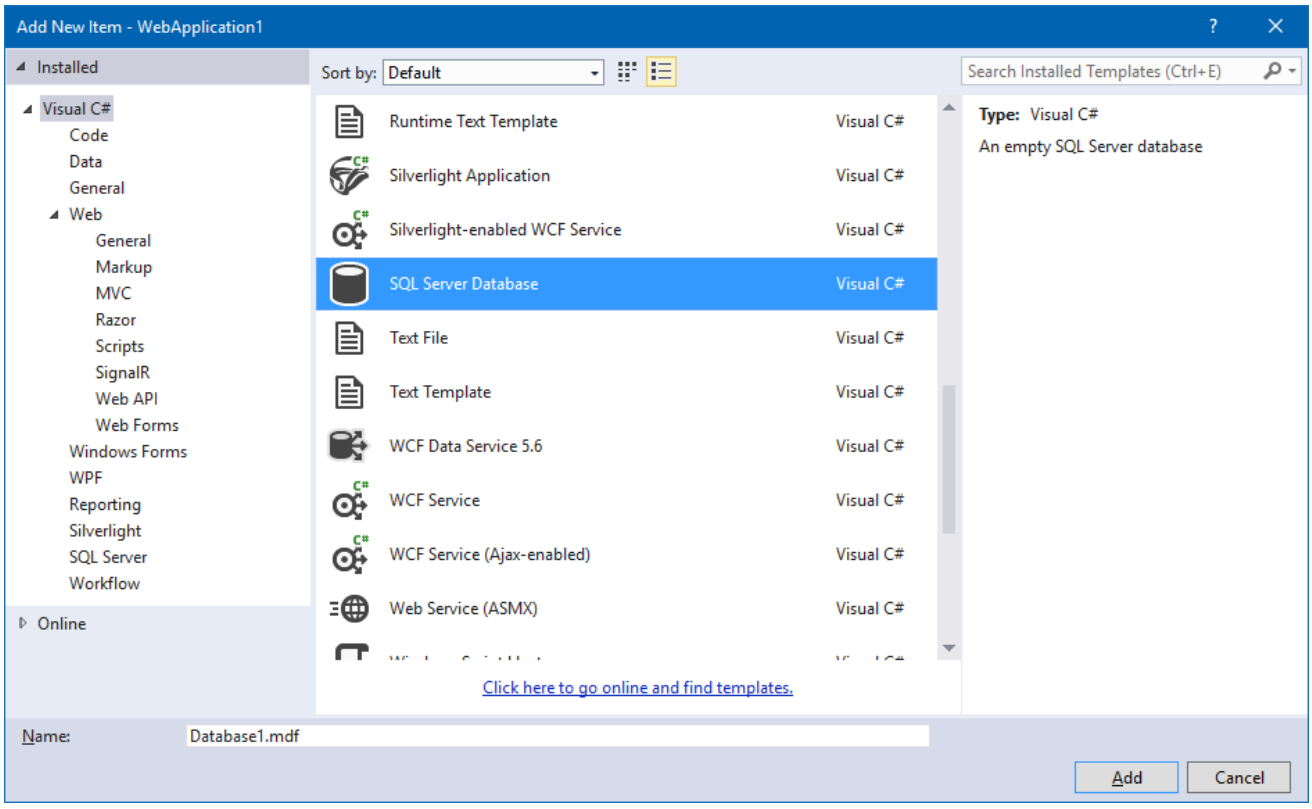
Para iniciar a criação de um banco de dados crie um novo projeto de web site vazio.



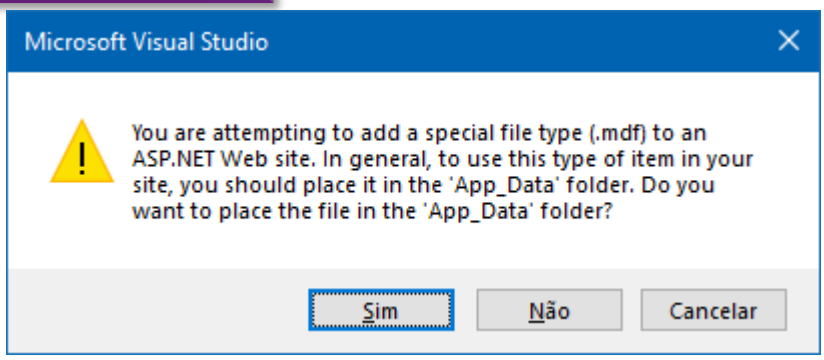
No painel Solution Explorer clique com o botão direito do mouse sobre o título do projeto e vá até **Add / New Item.**

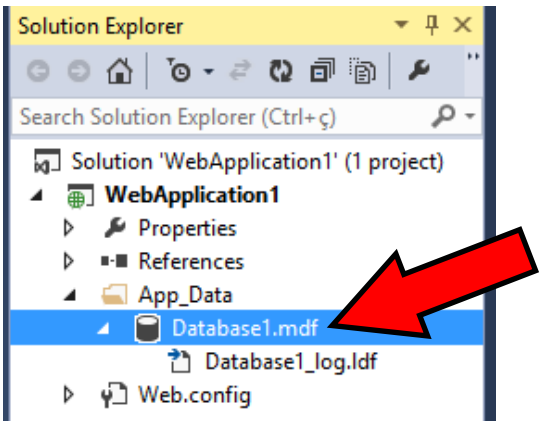


Na janela New Item escolha **SQL Server Database** e defina um nome para o banco de dados.

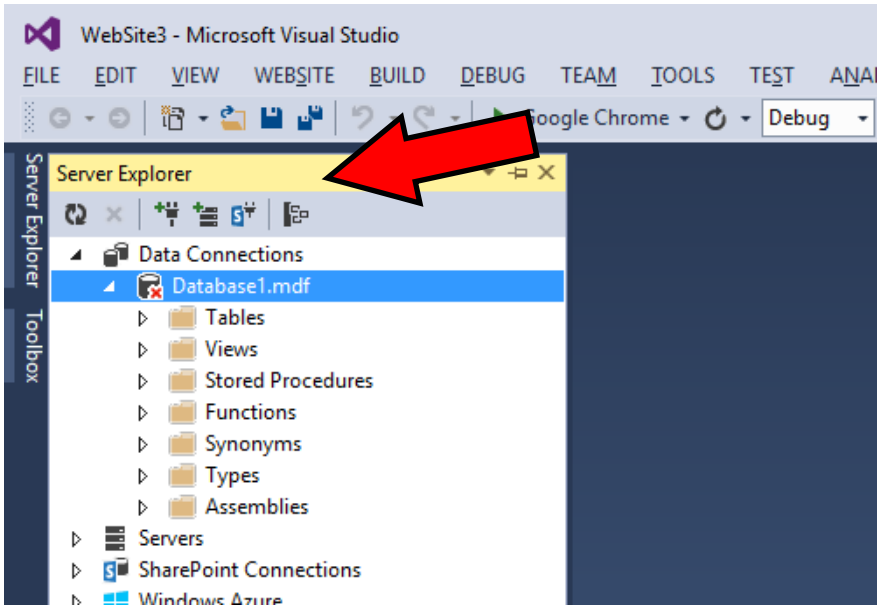


Uma caixa de diálogo aparecerá informando que o banco deverá ser alocado em uma pasta chamada App_Data. Confirme clicando em Sim para que o Visual Studio crie a pasta e salve o banco dentro dela.



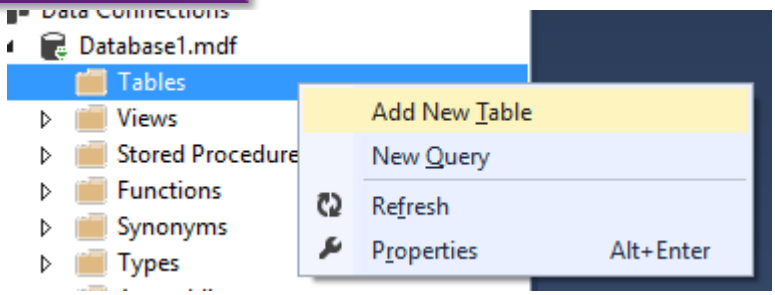


Do lado esquerdo da tela será exibida a aba para o painel **Server Explorer** permitindo a manipulação e o gerenciamento de bancos de dados.

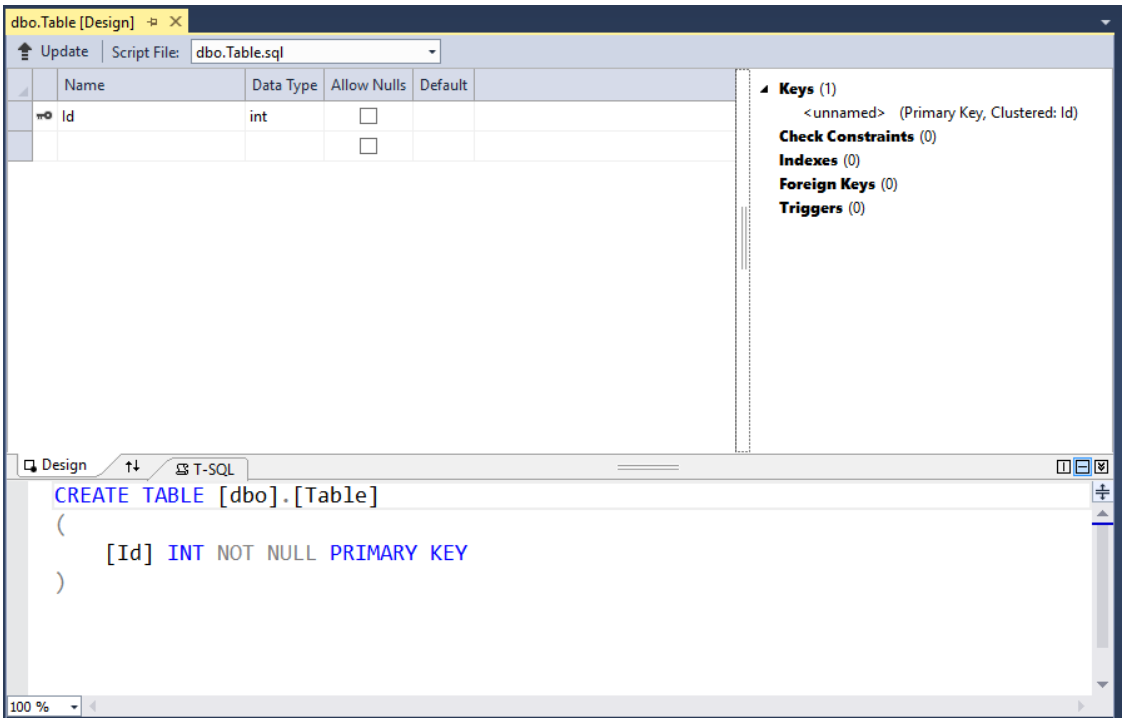


6.2. Criando tabelas

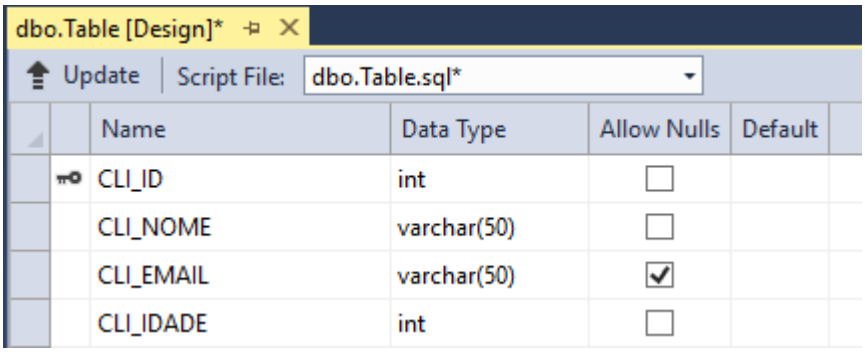
Para criar a primeira tabela para o banco criado clique com o botão direito do mouse na pasta **Tables / Add New Table** como mostra a figura a seguir:



Em seguida, na parte central da interface aparecerá uma tela em branco, dividida em Editor Gráfico e Editor de Scripts T-SQL na qual você poderá criar, tabelas, efetuar consultas, etc.



Na aba de Design devemos preencher os campos da tabela, selecionando também o seu tipo, e se ela aceita valore nulos.



Em sincronia, a aba do Editor de Scripts T-SQL é inserido o comando CREATE TABLE.

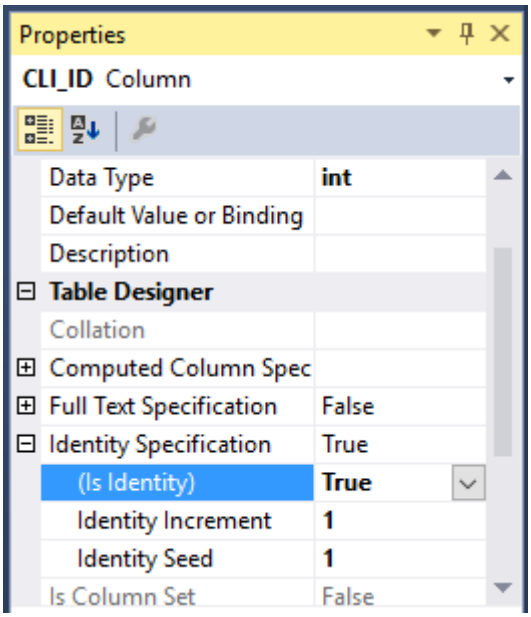
```
Design  T-SQL
CREATE TABLE [dbo].[Table]
(
    [CLI_ID] INT NOT NULL PRIMARY KEY,
    [CLI_NOME] VARCHAR(50) NOT NULL,
    [CLI_EMAIL] VARCHAR(50) NULL,
    [CLI_IDADE] INT NOT NULL
)
```

Para definir o nome da tabela altere o valor `[Table]` no comando CREATE TABLE no script para CLIENTE.

Defina a coluna CLI_ID como auto incremento para permitir o registro automático nessa coluna. Trata-se de um recurso nativo do SQL presente nas propriedades da coluna, no momento da criação da tabela.

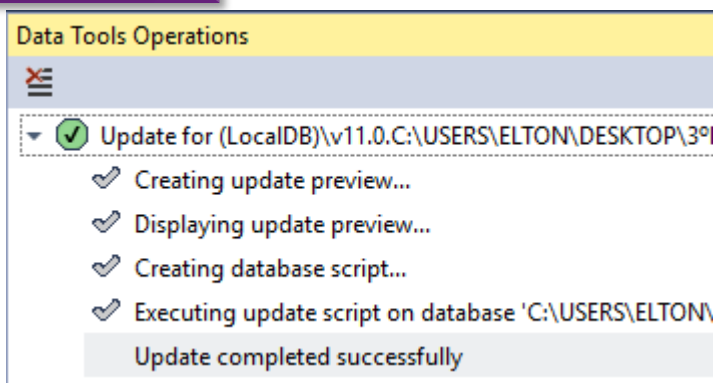
A propriedade IDENTITY é utilizada para atributos (campos/colunas) das tabelas nas funções CREATE TABLE e ALTER TABLE, e tem como finalidade incrementar um valor a cada nova inserção.

Clique na coluna CLI_ID e no painel Properties marque **True** para a propriedade **Identity Specification / Is Identity.**

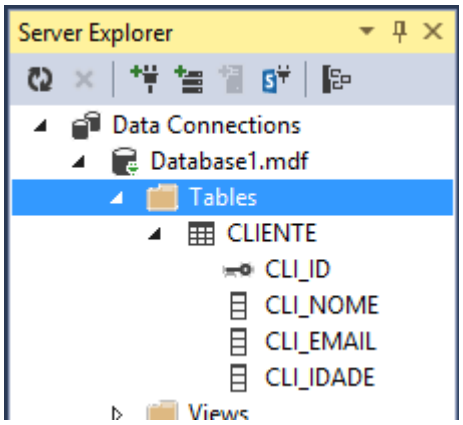


Para salvar a tabela CLIENTE clique no botão Update no topo da parte principal. Na janela que se abrirá chamada Preview Database Updates clique no botão **Update Database**.

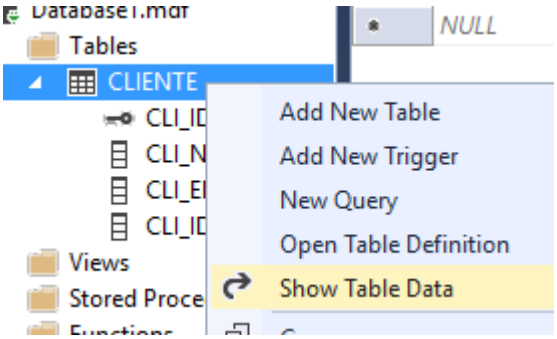
No painel Data Tools Operations será exibida a mensagem *Update completed successfully* indicando que a tabela foi salva com sucesso.



Clique com o botão direito do mouse na pasta Tables do painel Server Explorer e clique no botão **Refresh** para exibir a tabela criada.



Vamos, agora, inserir alguns registros para a nossa tabela recentemente criada. Para isso clique com o botão direito do mouse nela e clique em **Show Table Data.**



Insira os registros como mostra a figura a seguir:

dbo.CLIENTE [Data] | dbo.CLIENTE [Design]

Max Rows: 1000

	CLI_ID	CLI_NOME	CLI_EMAIL	CLI_IDADE
	1	Chimena de Souza	chisouza@hotmail.com	46
	2	Gumercindo de Oliveira	gu_oliveira@gmail.com	38
	3	Florisvaldo dos Santos	floris@hotmailcom	22
	4	João dos Santos	jao123@hotmail.com	18
	6	Filisbina Aparecida das Neves	fifi57@gmail.com	44
▶*	NULL	NULL	NULL	NULL

6.3. Exibir conteúdo de tabelas em um Web Form

Crie um Web Form e defina o nome como frmcliente. Vamos inserir um componente Grid View para permitir a visualização de tabelas do banco de dados. Para isso, localize-o no painel Toolbox localizado na categoria **Data** e insira no corpo do Web Form como mostra a figura abaixo:

frmcliente.aspx | dbo.CLIENTE [Data] | dbo.CLIENTE [Design]

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
    <div>
        <asp:GridView ID="GridView1" runat="server"></asp:GridView>
    </div>
    </form>
</body>
</html>
```

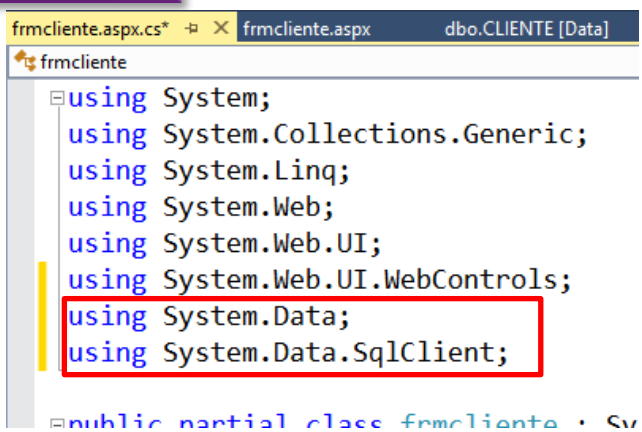
100 %

asp:GridView#GridView1

Column0	Column1	Column2
abc	abc	abc
abc	abc	abc
abc	abc	abc
abc	abc	abc
abc	abc	abc

Altere o ID do Grid View para **gvw_cliente**.

Vamos agora abrir o elemento frmcliente.aspx.cs para criar o script de conexão com o banco de dados. Para isso, teremos que fazer uso de duas classes: **System.Data** e **System.Data.SqlClient**. Essas deverão ser instanciadas no inicio do código como é mostrado a seguir:



Para dar início, devemos criar o código de conexão dentro do evento Page_Load:

```
SqlConnection conecta = new SqlConnection();
```

Entre o parênteses de SqlConnection deve ser inserida a string de conexão com o banco que foi criado. Para isso, vá até o painel Server Explorer, clique no nome do banco de dados criado, depois vá ao painel Properties e copie o valor da propriedade **Connection String** e cole no código entre aspas como mostra abaixo:

```
SqlConnection conecta = new SqlConnection("Data
Source=(LocalDB)\v11.0;AttachDbFilename=C:\Users\elton\WebSite3\App_Data\Database1.
mdf;Integrated Security=True");
```

Irá ocorrer um erro no código devido as barras no endereço do arquivo do banco. Para corrigir, insira um @ no início da string de conexão:

```
SqlConnection conecta = new SqlConnection(@"Data
Source=(LocalDB)\v11.0;AttachDbFilename=C:\Users\elton\WebSite3\App_Data\Database1.
mdf;Integrated Security=True");
```

Próximo passo e realizar a abertura da conexão inserindo o código abaixo na próxima linha:

```
conecta.Open();
```

Vamos agora criar um comando adaptador de dados que vai listar os registros da tabela:

```
SqlDataAdapter adp = new SqlDataAdapter("SELECT * FROM CLIENTE", conecta);
```

Em sequência devemos criar o comando **DataSet** que é responsável para armazenar e mover dados de um banco de dados, mas o próprio DataSet não faz conexão alguma com o banco, ele nem mesmo tem um objeto para realizar tal conexão. Segue abaixo:

```
DataSet dset = new DataSet();
```

A ligação entre o banco de dados e DataSet é feita pelo objeto DataAdapter. E para realizar essa ligação utilize o comando:

```
adp.Fill(dset);
```

Após isso, vamos realizar a ligação com o Grid View:

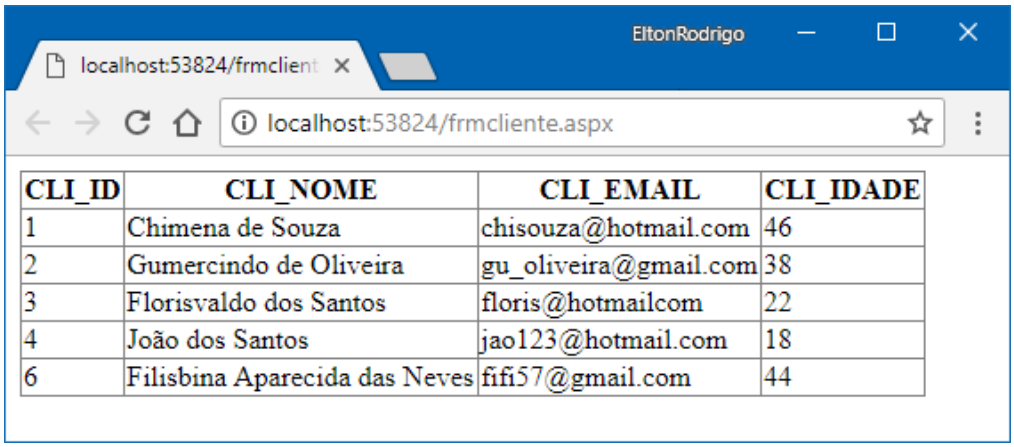
```
gvw_cliente.DataSource = dset;
gvw_cliente.DataBind();
```

Em resumo, todo o script ficou assim:

```
SqlConnection conecta = new SqlConnection(@"Data
Source=(LocalDB)\v11.0;AttachDbFilename=C:\Users\elton\WebSite3\App_Data\Dat
abase1.mdf;Integrated Security=True");
conecta.Open();
SqlDataAdapter adp = new SqlDataAdapter("SELECT * FROM CLIENTE", conecta);
DataSet dset = new DataSet();
adp.Fill(dset);

gvw_cliente.DataSource = dset;
gvw_cliente.DataBind();
```

Pronto! Execute o projeto no navegador e o resultado será o que está mostrando abaixo:

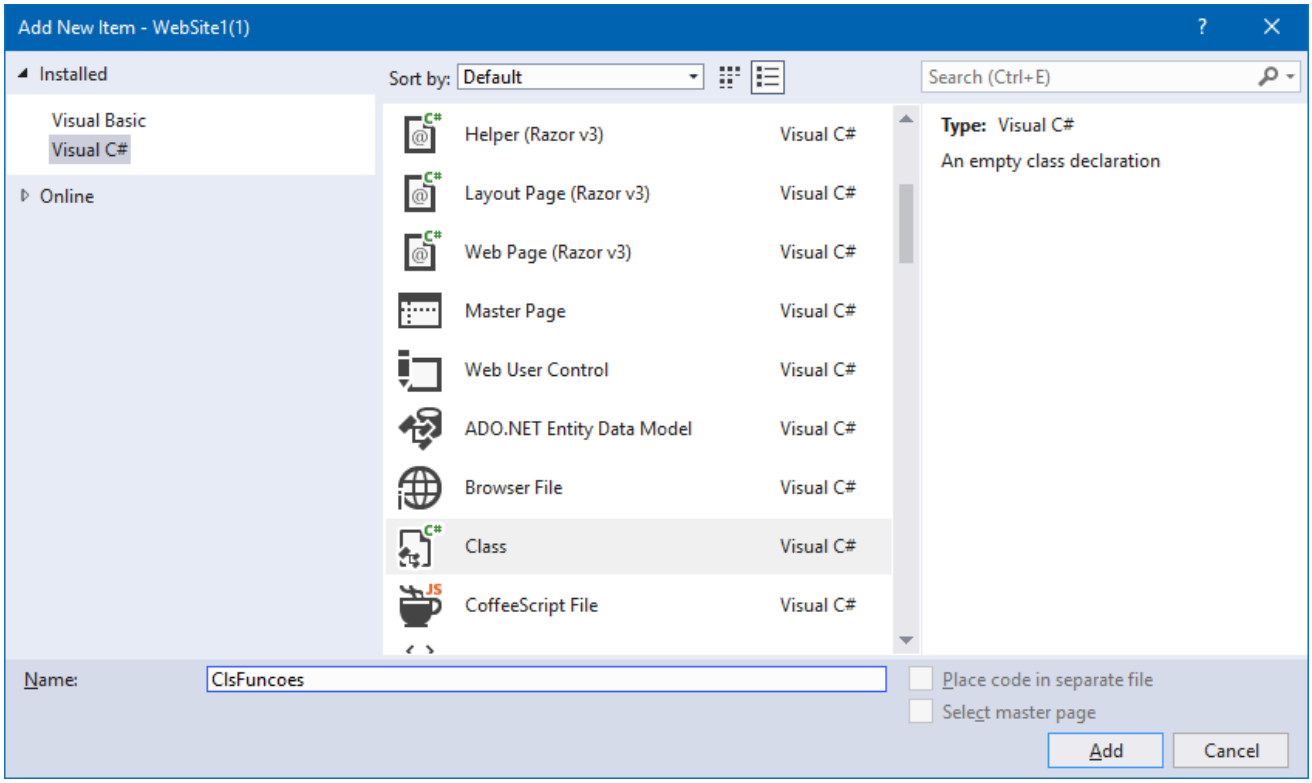


CLI_ID	CLI_NOME	CLI_EMAIL	CLI_IDADE
1	Chimena de Souza	chisouza@hotmail.com	46
2	Gumercindo de Oliveira	gu_oliveira@gmail.com	38
3	Florisvaldo dos Santos	floris@hotmailcom	22
4	João dos Santos	jao123@hotmail.com	18
6	Filisbina Aparecida das Neves	fifi57@gmail.com	44

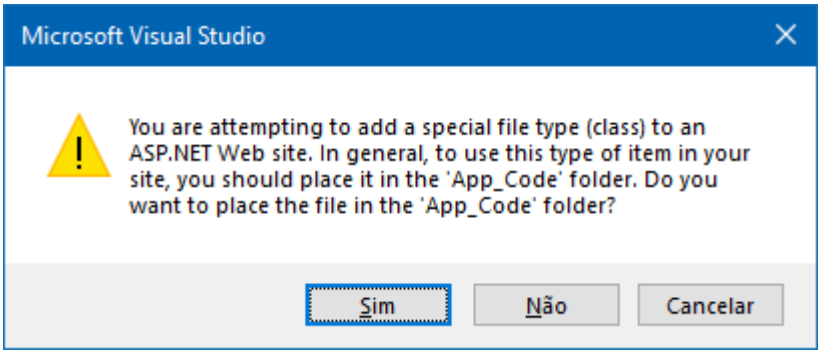
6.4. Organizando o código

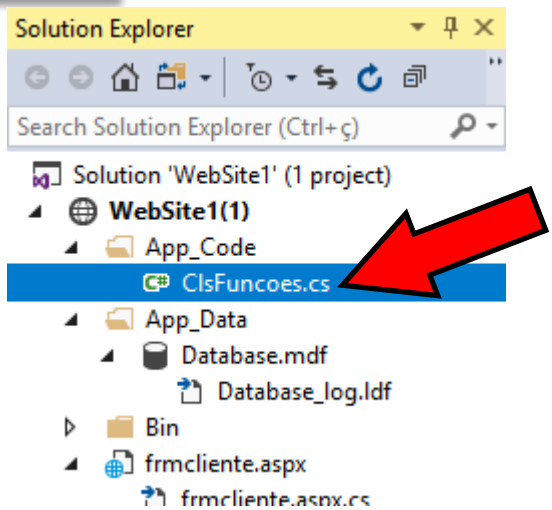
Faz se necessário realizar a organização do código utilizando técnicas de orientação a objetos utilizando métodos e classes.

No painel Solution Explorer clique com o botão direito do mouse sobre o título do projeto e vá até **Add / New Item**. Na janela New Item escolha **Class** e defina o nome para a classe como **ClsFuncoes**.

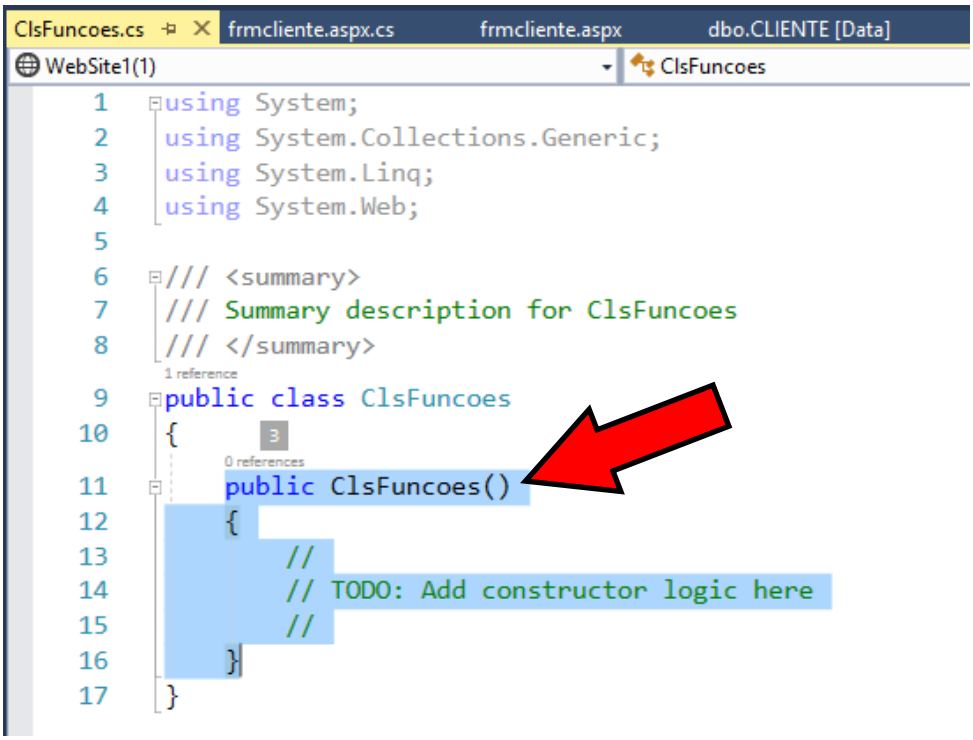


Uma caixa de diálogo aparecerá informando que a classe deverá ser alocada em uma pasta chamada **App_Code**. Confirme clicando em Sim para que o Visual Studio crie a pasta e salve a classe dentro dela.





Dentro do código da class ClsFuncoes automaticamente é criado o bloco com o mesmo nome. Exclua o bloco public ClsFuncoes.



Do mesmo modo, para fazer uso das funções do SQL é necessário fazer uso das classes: **System.Data** e **System.Data.SqlClient**. Essas deverão ser instanciadas no início do código como é mostrado a seguir:

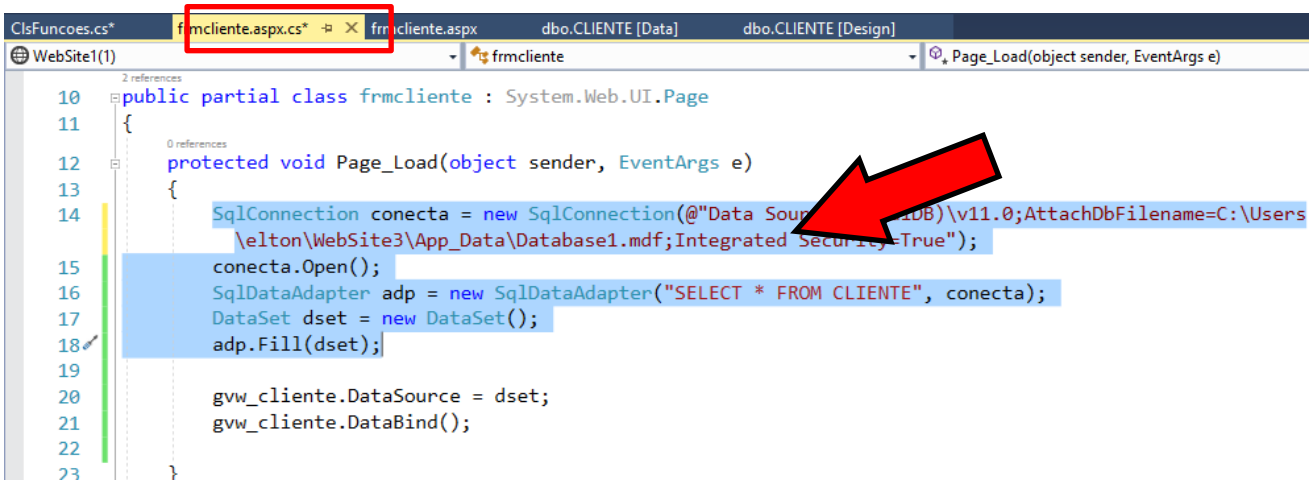
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Data;
using System.Data.SqlClient;
```

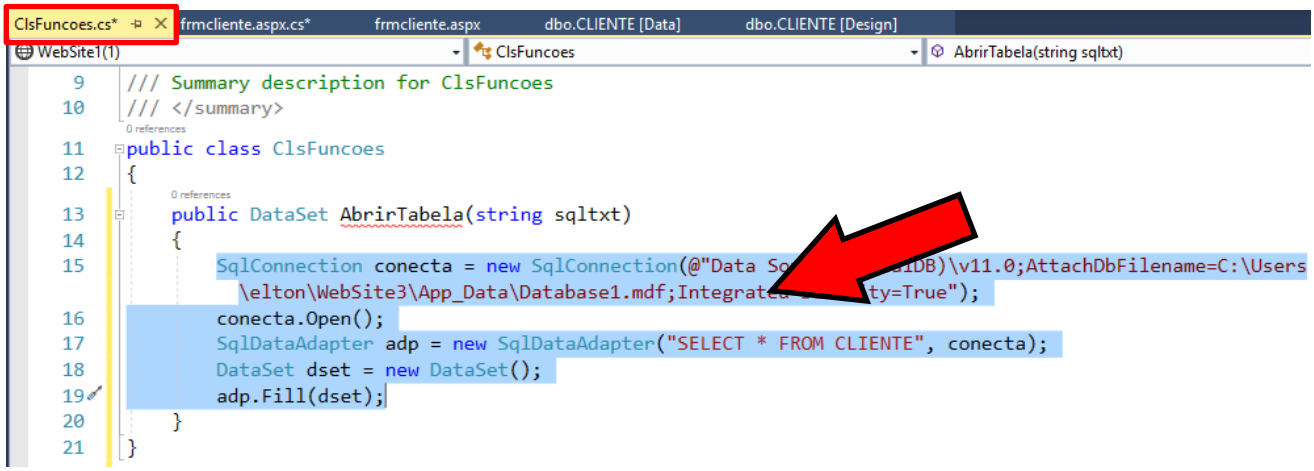
Vamos criar o método do DataSet para abrir a tabela do banco de dados. O DataSet terá o nome **AbrirTabela** com uma string chamada **sqltxt**.

```
public class ClsFuncoes
{
    0 references
    public DataSet AbrirTabela(string sqltxt)
    {

    }
}
```

Copie o script do evento Page_Load do frmcliente e cole dentro do bloco do método AbrirTabela da class ClsFuncoes.





Vamos passar como parâmetro o método sqltxt na função do SqlDataAdapter como mostra a figura abaixo:

```
public DataSet AbrirTabela(string sqltxt)
{
    SqlConnection conecta = new SqlConnection(@"Data Source=(Loca
      \elton\WebSite3\App_Data\Database1.mdf;Integrated Security=
    conecta.Open();
    SqlDataAdapter adp = new SqlDataAdapter(sqltxt, conecta);
    DataSet dset = new DataSet();
    adp.Fill(dset);
```

Dessa forma, quando o método AbrirTabela for chamado o comando SELECT do SQL será incluído no sqltxt e deverá retornar no DataSet. Portanto é necessário incluir o comando para isso.

```
public DataSet AbrirTabela(string sqltxt)
{
    SqlConnection conecta = new SqlConnection(@"Data Source=(Loc
      \elton\WebSite3\App_Data\Database1.mdf;Integrated Security=
    conecta.Open();
    SqlDataAdapter adp = new SqlDataAdapter(sqltxt, conecta);
    DataSet dset = new DataSet();
    adp.Fill(dset);
    return dset;
}
```

Volte agora para o arquivo frmcliente, deixe em forma de comentário o script copiado para a classe ClsFuncoes.

```
protected void Page_Load(object sender, EventArgs e)
{
    //SqlConnection conecta = new SqlConnection(@"Data Source=(LocalDB)\v11.0;AttachDbFilename=C:\Users
      \elton\WebSite3\App_Data\Database1.mdf;Integrated Security=True");
    //conecta.Open();
    //SqlDataAdapter adp = new SqlDataAdapter("SELECT * FROM CLIENTE", conecta);
    //DataSet dset = new DataSet();
    //adp.Fill(dset);

    gvw_cliente.DataSource = dset;
    gvw_cliente.DataBind();

}
```

Substitua agora a chamada do gridview alterando para a classe criada.

```
gvw_cliente.DataSource = new ClsFuncoes().AbrirTabela("SELECT * FROM CLIENTE");
gvw_cliente.DataBind();
```

6.5. Inserir registros a partir de um formulário

No WebForm crie os campos de texto para os campos da tabela cliente e um botão seguindo as propriedades abaixo:

```
<body>
    <form id="form1" runat="server">

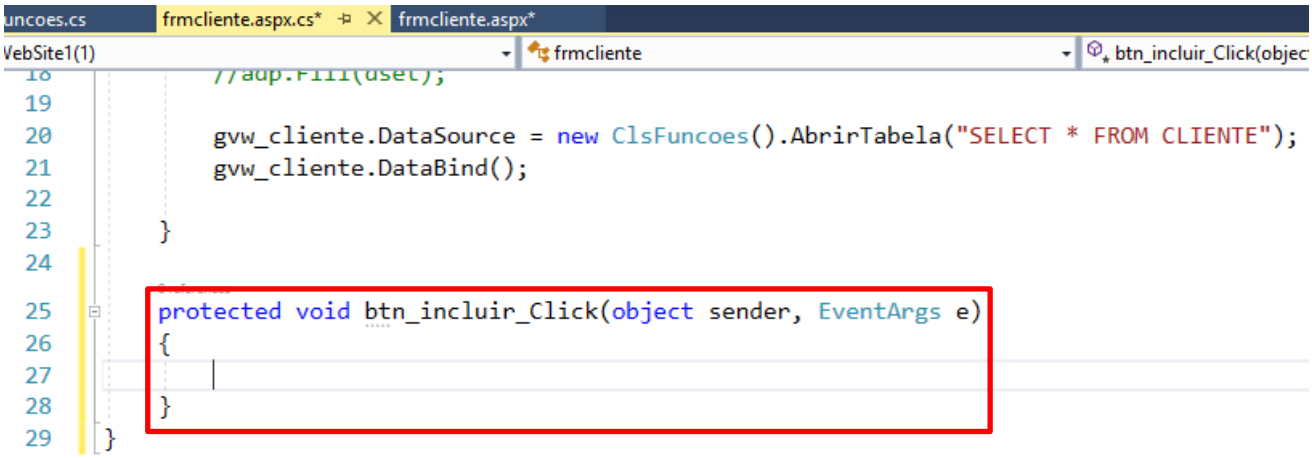
        <label>Nome: </label> <asp:TextBox ID="txt_nome" runat="server" Width="400"></asp:TextBox>
        <br />
        <label>Email: </label> <asp:TextBox ID="txt_email" runat="server" Width="300"></asp:TextBox>
        <br />
        <label>Idade: </label> <asp:TextBox ID="txt_idade" runat="server" Width="50"></asp:TextBox>
        <br />
        <asp:Button ID="btn_incluir" runat="server" Text="Button" />

        <div>
            <asp:GridView ID="gvw_cliente" runat="server"></asp:GridView>
        </div>

    </form>
</body>
```



Vamos criar agora o evento do botão do formulário, para isso dê um duplo clique nele e será aberto o arquivo de script com o evento criado.



Próximo passo é criar a conexão com o banco de dados criando uma variável como objeto para receber a função ClsFuncoes da class e atribuindo esse objeto ao SqlConnection:

```
var obj = new ClsFuncoes();
SqlConnection conecta = new SqlConnection(obj.conexao);
conecta.Open();
```

Para dar início ao script de armazenamento de dados no banco vamos precisar usar o comando sql INSERT passando por parâmetros de TextBox.

```
string sqltxt = string.Format("INSERT INTO CLIENTE(CLI_NOME,CLI_EMAIL,CLI_IDADE)
VALUES('{0}','{1}','{2}')",txt_nome.Text,txt_email.Text,txt_idade.Text);
```

Depois de armazenado na memória precisamos executar o comando para incluir esse script no banco de dados através do **SqlCommand.**

```
SqlCommand cmd = new SqlCommand(sqltxt, conecta);
cmd.ExecuteNonQuery();
```

Após os registros inseridos é necessário limpar os campos de texto:

```
txt_nome.Text = "";
txt_email.Text = "";
txt_idade.Text = "";
```

Para finalizar é preciso recarregar a gridview clientes para ser exibida na página novamente.

```
gvw_cliente.DataSource = new ClsFuncoes().AbrirTabela("SELECT * FROM
CLIENTE");
gvw_cliente.DataBind();
```

Em resumo, segue abaixo todo o script para o botão incluir:

```
SqlConnection conecta = new SqlConnection @"Data
Source=(LocalDB)\v11.0;AttachDbFilename=C:\Users\elton\WebSite3\App_Data\Dat
abase1.mdf;Integrated Security=True");
conecta.Open();

string sqltxt = string.Format("INSERT INTO
CLIENTE(CLI_NOME,CLI_EMAIL,CLI_IDADE)
VALUES('{0}','{1}','{2}')",txt_nome.Text,txt_email.Text,txt_idade.Text);

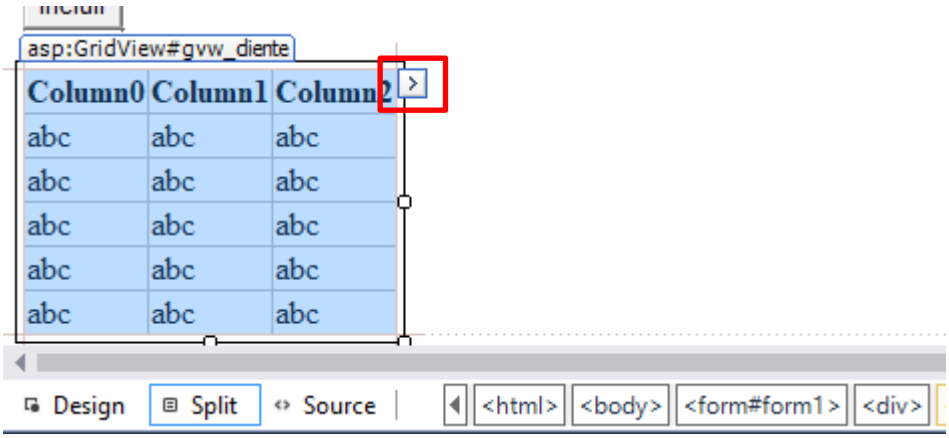
SqlCommand cmd = new SqlCommand(sqltxt, conecta);
cmd.ExecuteNonQuery();
txt_nome.Text = "";
txt_email.Text = "";
txt_idade.Text = "";

gvw_cliente.DataSource = new ClsFuncoes().AbrirTabela("SELECT * FROM
CLIENTE");
gvw_cliente.DataBind();
```

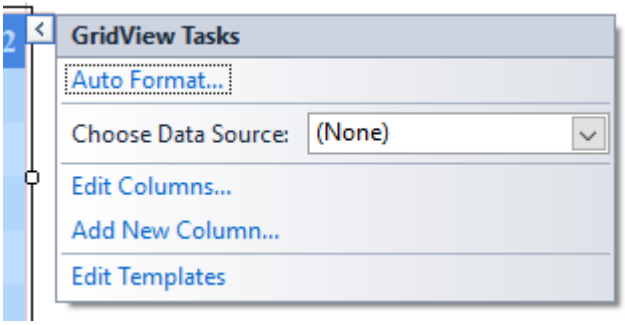
6.6. Selecionar registros pela GridView

É possível utilizar um recurso do Visual Studio para selecionar dados na GridView e carrega-los nos TextBoxs.

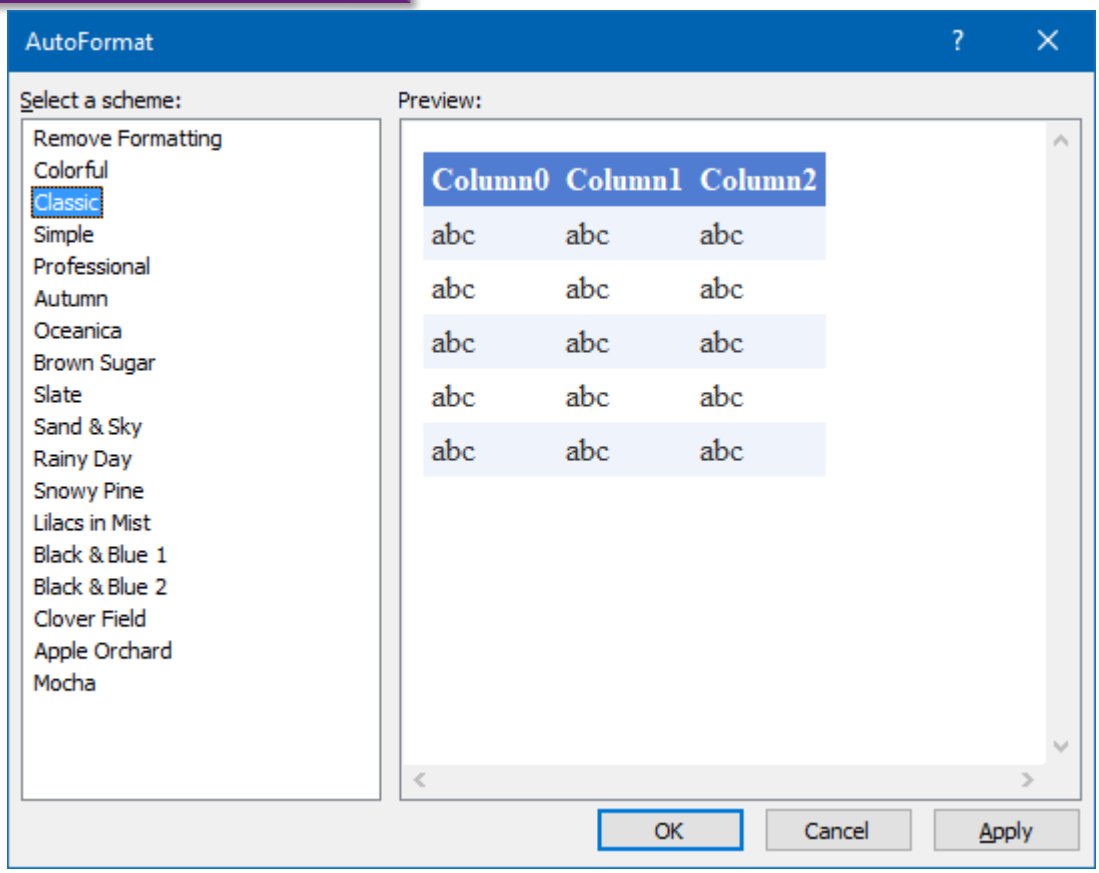
Selecione a GridView já inserida através do modo de exibição Design e clique na seta do campo superior direito:



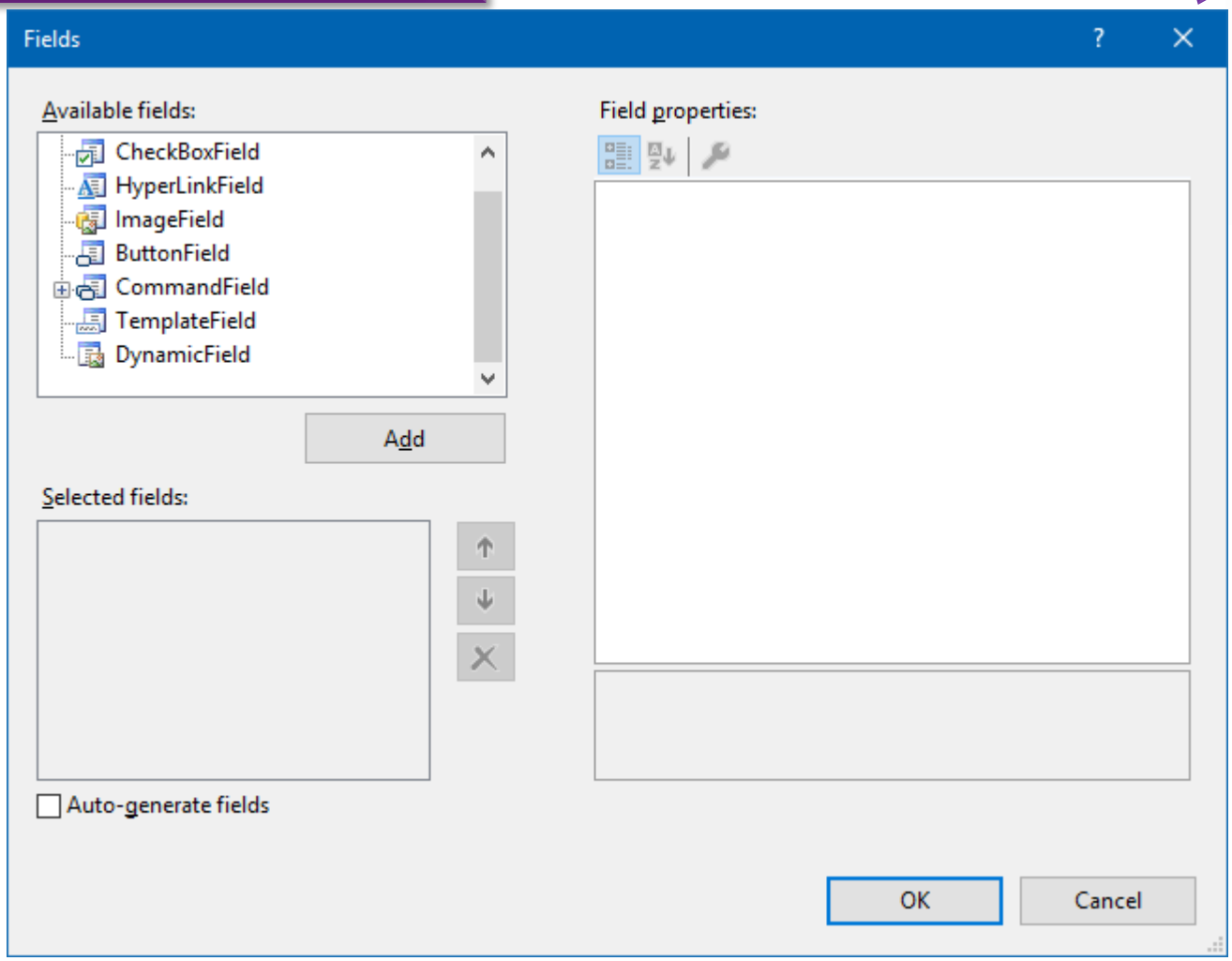
Será aberto um menu chamado **GridView Tasks**. Nesse menu clique em **Auto Format**.



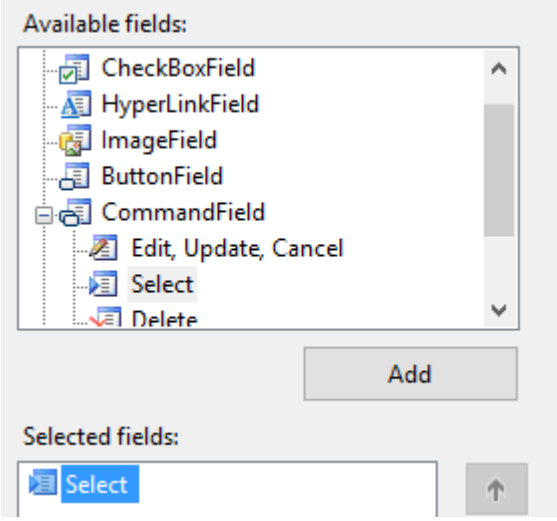
Aqui é possível escolher a aparência da tabela selecionando um dos esquemas oferecidos.



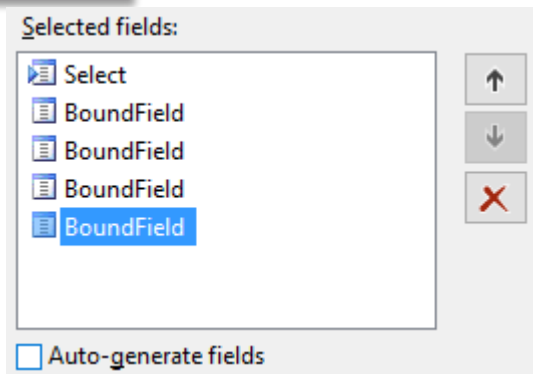
Voltando no menu **GridView Tasks** escolha agora a opção **Edit Columns,** será aberta uma janela chamada **Fields** (Campos).



Na área **Available fields** expanda o item **CommandField,** clique no sub item **Select** e clique em **Add.**



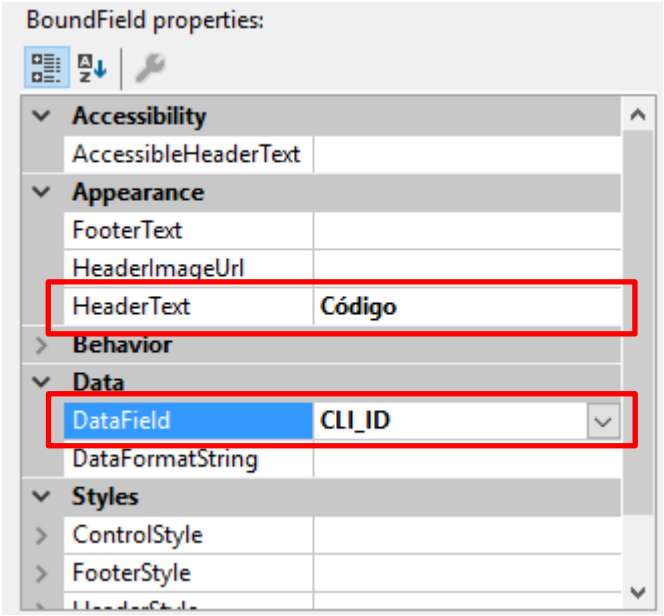
Adicione também o item **BoundField** 4 vezes:



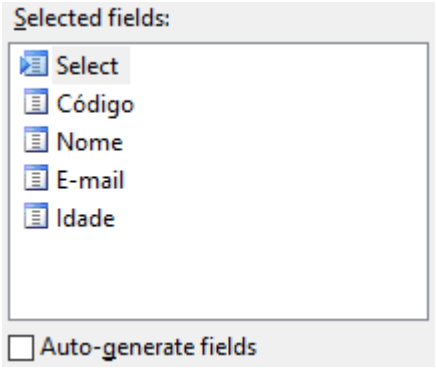
Desmarque a opção **Auto-generate fields**.

Dentro da caixa **Selected fields** clique no item **Select** e escolha sua aparência na categoria **Appearance**.

Para cada **BoundField** escolha um texto que representa cada coluna da tabela CLIENTES do banco de dados no campo **HeaderText** e informe o nome da coluna em **DataField** na categoria **Data** e clique em OK.



Cada **Bound field** terá o nome de cada coluna que foi definida.



Dê um duplo clique na GridView para carregar um procedimento para ela dentro do script.

```
0 references
protected void gvw_cliente_SelectedIndexChanged(object sender, EventArgs e)
{

}
```

Vamos criar um método que ao selecionar uma linha irá passar para o parâmetro do código do cliente. É necessário criar uma variável que receberá o código do registro de cada linha.

```
string codcli = gvw_cliente.SelectedRow.Cells[1].Text;
```

Em SelectedRow.Cells[1], o número entre os colchetes indica o número da coluna COD_CLI da tabela Clientes.

O próximo passo é criar um método que retornará um dataset responsável por carregar os campos da tabela.

```
0 references
DataSet Cliente(string _codigo)
{

}
```

Para ter acesso a tabela inserimos a conexão primeiro.

```
var obj = new ClsFuncoes();
SqlConnection conecta = new SqlConnection(obj.conexao);
conecta.Open();
```

Criamos, então, um dataset para realizar a consulta na tabela atribuindo um parâmetro @COD na cláusula where, possibilitando a seleção de cada registro.

```
DataSet _dst = new DataSet();
SqlCommand cmd = new SqlCommand("SELECT * FROM CLIENTE WHERE CLI_ID=@COD",conecta);
cmd.Parameters.AddWithValue("@COD", _codigo);
```

Fechando esse método é necessário inserir o SqlDataAdapter e retornar o valor do Dataset.

```
SqlDataAdapter adp = new SqlDataAdapter(cmd);
adp.Fill(_dst);
return _dst;
```

Lembrando que a propriedade Fill não irá funcionar se não houver o parâmetro de conexão na função SqlCommand.

Voltando ao procedimento da gridview, criamos as variáveis que irão armazenar os dados que serão selecionados na tabela.

```
0 references
protected void gvw_cliente_SelectedIndexChanged(object sender, EventArgs e)
{
    string codcli = gvw_cliente.SelectedRow.Cells[1].Text;
    var cli = Cliente(codcli).Tables[0];
    txt_nome.Text = cli.Rows[0]["CLI_NOME"].ToString();
    txt_email.Text = cli.Rows[0]["CLI_EMAIL"].ToString();
    txt_idade.Text = cli.Rows[0]["CLI_IDADE"].ToString();
}
```

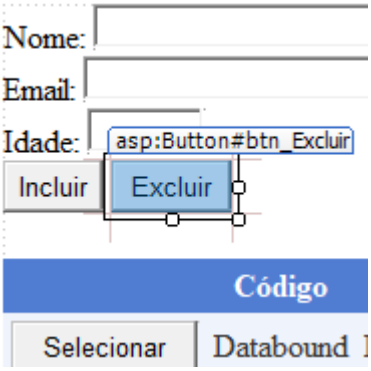
Após compilar o código repare que para cada linha haverá um botão chamado Selecionar. Clicando em um deles a linha será selecionada e os dados serão exibidos nos campos de texto.

Nome: Gumercindo de Oliveira
Email: gu_oliveira@gmail.com
Idade: 38
Incluir

	Código	Nome	E-mail	Idade
Selecionar	1	Chimena de Souza	chisouza@hotmail.com	46
Selecionar	**2**	**Gumercindo de Oliveira**	**gu_oliveira@gmail.com**	**38**
Selecionar	3	Florisvaldo dos Santos	floris@hotmail.com	22
Selecionar	4	João dos Santos	jao123@hotmail.com	18
Selecionar	5	Filisbina Aparecida das Neves	fifi57@gmail.com	44

6.7. Exclusão de registros

Insira um elemento Button com a propriedade Text com o nome Excluir no webform ao lado do botão Incluir.



Dê um duplo clique no botão para criar um procedimento de button_click no script.

```
0 references
protected void Button1_Click(object sender, EventArgs e)
{

}
```

Vamos realizar novamente a conexão com o banco dentro desse bloco.

```
var obj = new ClsFuncoes();
SqlConnection conecta = new SqlConnection(obj.conexao);
conecta.Open();
```

Criamos agora uma variável que receberá o SqlCommand que dessa vez executará a linha para exclusão de registros.

```
SqlCommand cmd = new SqlCommand("DELETE FROM CLIENTE WHERE CLI_ID=@COD", conecta);
```

Ainda é necessário atribuir o comando criado para a exclusão da linha selecionada.

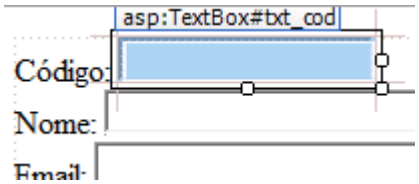
```
string codigo = gvw_cliente.SelectedRow.Cells[1].Text;
cmd.Parameters.AddWithValue("@COD",codigo);
cmd.ExecuteNonQuery();
```

Pronto. Agora é só inserir o carregamento da gridview novamente.

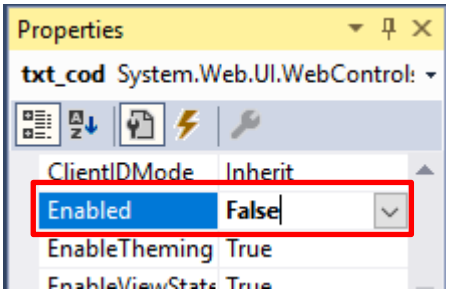
```
gvw_cliente.DataSource = new ClsFuncoes().AbrirTabela("SELECT * FROM CLIENTE");
gvw_cliente.DataBind();
```

6.8. Alteração de registros

Antes de inserir o botão responsável pela alteração dos registros, vamos adicionar um TextBox para o campo CLI_ID no webform com o ID **txt_cod**.



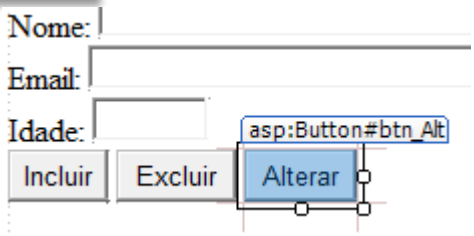
Altere a propriedade **Enabled** desse TextBox para **False**, para que o mesmo não seja alterado pelo usuário.



No procedimento de seleção de registros criado para gridview adicione a linha que irá exibir o ID do cliente.

```
protected void gvw_cliente_SelectedIndexChanged(object sender, EventArgs e)
{
    string codcli = gvw_cliente.SelectedRow.Cells[1].Text;
    var cli = Cliente(codcli).Tables[0];
    txt_cod.Text = cli.Rows[0]["CLI_ID"].ToString();
    txt_nome.Text = cli.Rows[0]["CLI_NOME"].ToString();
    txt_email.Text = cli.Rows[0]["CLI_EMAIL"].ToString();
    txt_idade.Text = cli.Rows[0]["CLI_IDADE"].ToString();
}
```

Insira um elemento Button com a propriedade Text com o nome **Alterar** no webform ao lado do botão Excluir.



Dê um duplo clique no botão para criar um procedimento de button_click no script.

```
0 references
protected void btn_Alt_Click(object sender, EventArgs e)
{

}
```

Vamos realizar novamente a conexão com o banco dentro desse bloco.

```
var obj = new ClsFuncoes();
SqlConnection conecta = new SqlConnection(obj.conexao);
conecta.Open();
```

Criamos agora mais uma variável que receberá o SqlCommand que dessa vez executará a linha para alteração de registros.

```
SqlCommand cmd = new SqlCommand("UPDATE CLIENTE SET CLI_NOME=@NOME, CLI_EMAIL=@EMAIL,
CLI_IDADE=@IDADE WHERE CLI_ID=@COD", conecta);
```

Faz-se necessário atribuir o comando criado para a alteração da linha selecionada utilizando o parâmetro **AddWithValue** novamente para cada TextBox.

```
cmd.Parameters.AddWithValue("@COD", txt_cod.Text);
cmd.Parameters.AddWithValue("@NOME", txt_nome.Text);
cmd.Parameters.AddWithValue("@EMAIL", txt_email.Text);
cmd.Parameters.AddWithValue("@IDADE", txt_idade.Text);

cmd.ExecuteNonQuery();
```

Pronto. Agora é só inserir o carregamento da gridview novamente.

```
gvw_cliente.DataSource = new ClsFuncoes().AbrirTabela("SELECT * FROM CLIENTE");
gvw_cliente.DataBind();
```